Understanding Artificial Intelligence: From Mathematics to Applications

Lecture 7:
Deep learning

Jean Feng

Announcements

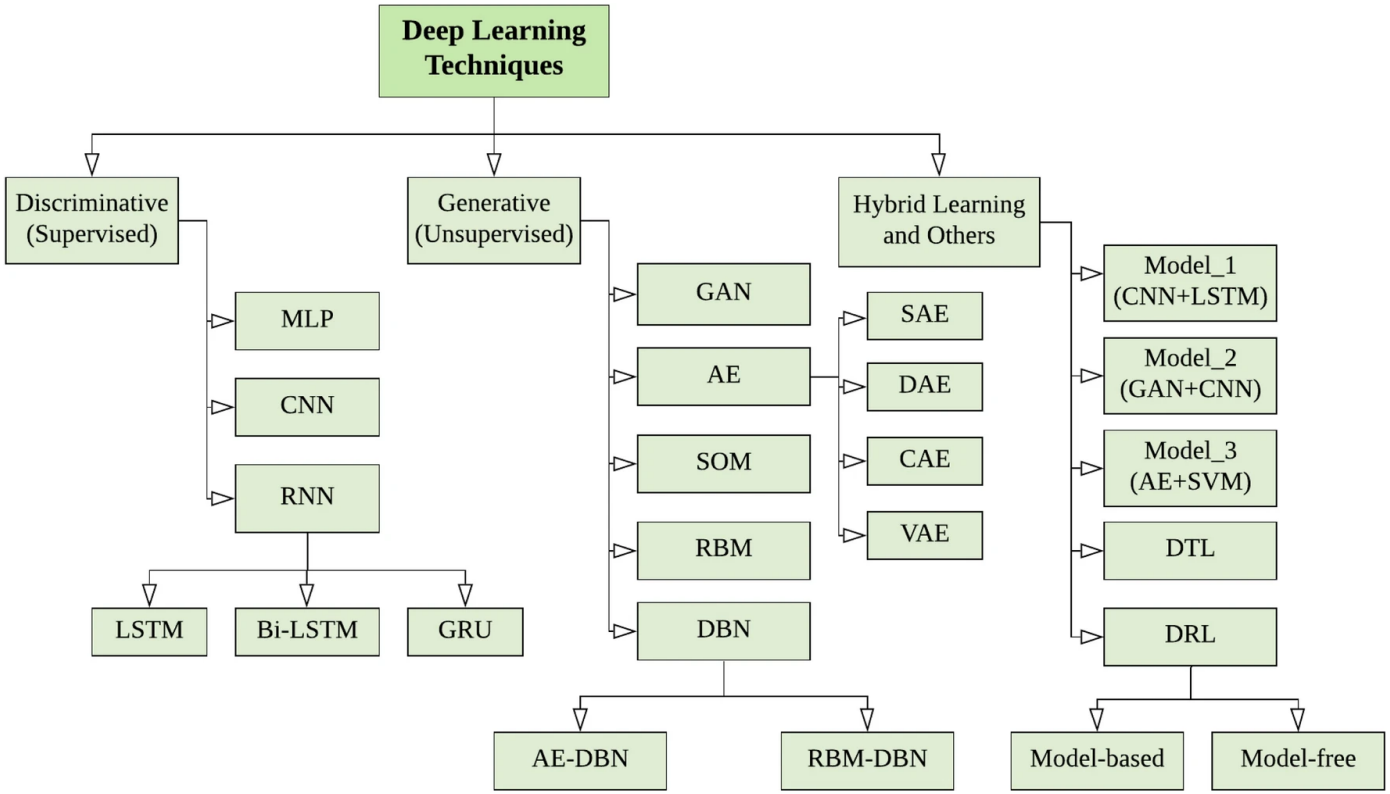
- HW3 is due in a week.
- Get started on your projects. Project check-in is due 5/25.
- HW4 is available now. Also due 5/25.

Lecture Outline

- **Deep learning**
- Dense neural networks
  - Backward Propagation
  - Optimization: From Stochastic Gradient Descent to Adam
  - Numerical Stability and Initialization
  - Bias-variance tradeoff
- Pytorch

Deep learning

Q: What are some deep learning models/architectures you have heard about?



Sarker 2021

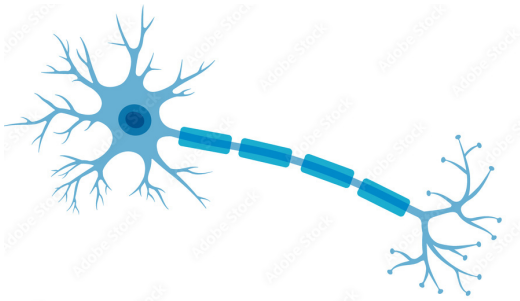
Deep learning: A brief history

Bulletin of Mathematical Biology Vol. 52, No. 1/2, pp. 99–115, 1990.
Printed in Great Britain.

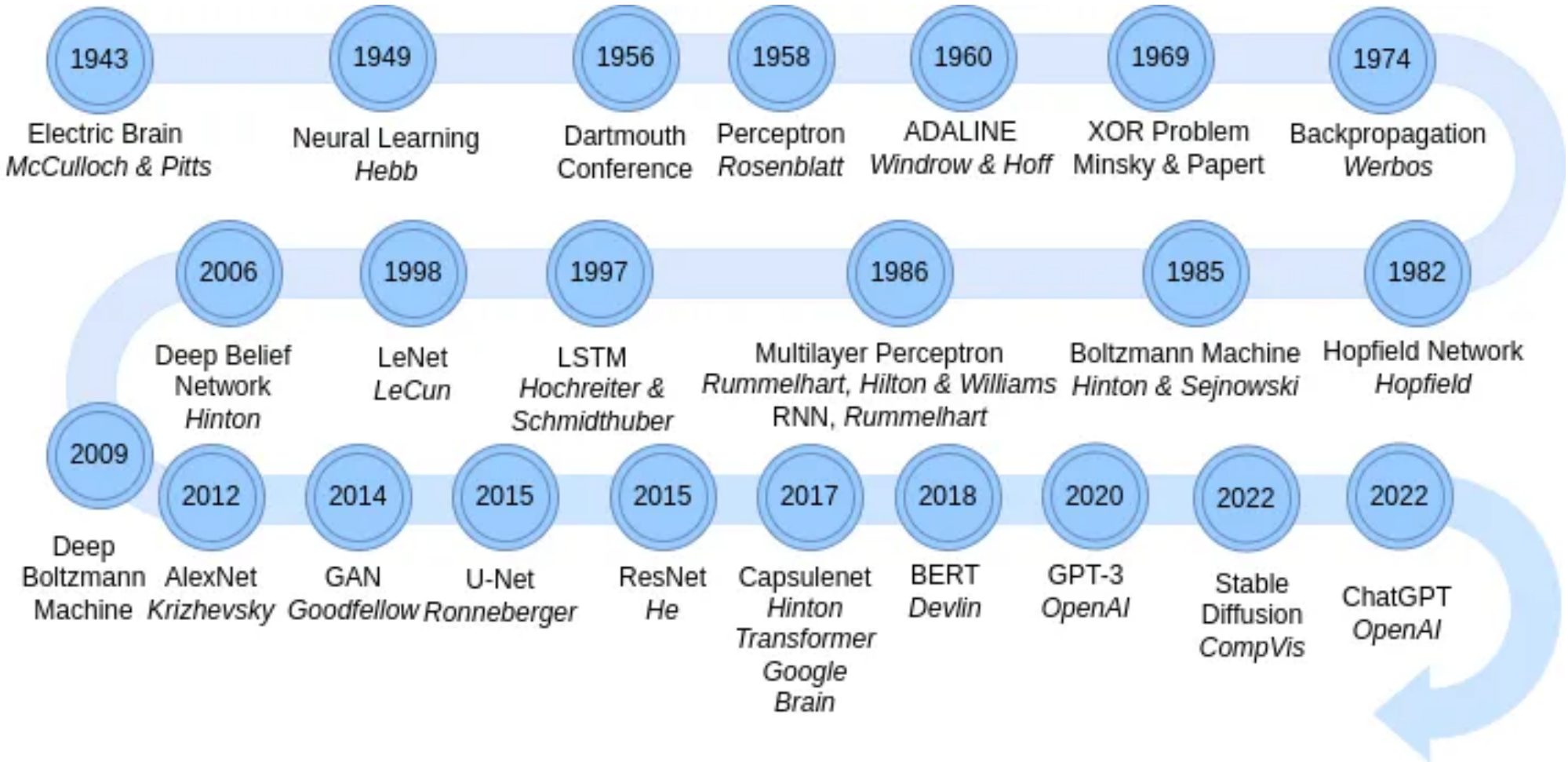
0092-8240/90$3.00 + 0.00
Pergamon Press plc
Society for Mathematical Biology

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

■ WARREN S. MCCULLOCH AND WALTER PITTS
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.



Deep learning: A brief history



Deep learning: today

You
Provide a brief history of deep learning

ChatGPT
Deep learning is a subset of machine learning in artificial intelligence (AI) that has networks capable of learning unsupervised from data that is unstructured or unlabeled. Here's a brief overview of the history and development of deep learning:

- **1950s - Early Concepts**: The roots of deep learning can be traced back to the 1950s, with the introduction of the perceptron by Frank Rosenblatt. It was an early attempt to model the neurons in the human brain.
- **1960s - Neural Networks**: The concept of neural networks began to take shape, with the development of the ADALINE (ADAptive LInear NEuron) by Bernard Widrow and Ted Hoff. However, limitations such as the XOR problem, where a single-layer perceptron couldn't solve non-linearly separable problems, led to a reduction in neural network research enthusiasm.
- **1980s - Resurgence & Backpropagation**: Interest in neural networks was revived in

You
make me a cartoon photo of a cute poodle

ChatGPT

Here's a cartoon photo of a cute poodle for you! I hope you find it as adorable as intended.

What's so deep about deep learning

- Deep learning uses multiple layers/modules to progressively extract higher-level features from the raw input.
  - Each module is composed of multiple layers.
  - Each layer is composed of multiple nodes.
  - Each node is associated with multiple parameters.

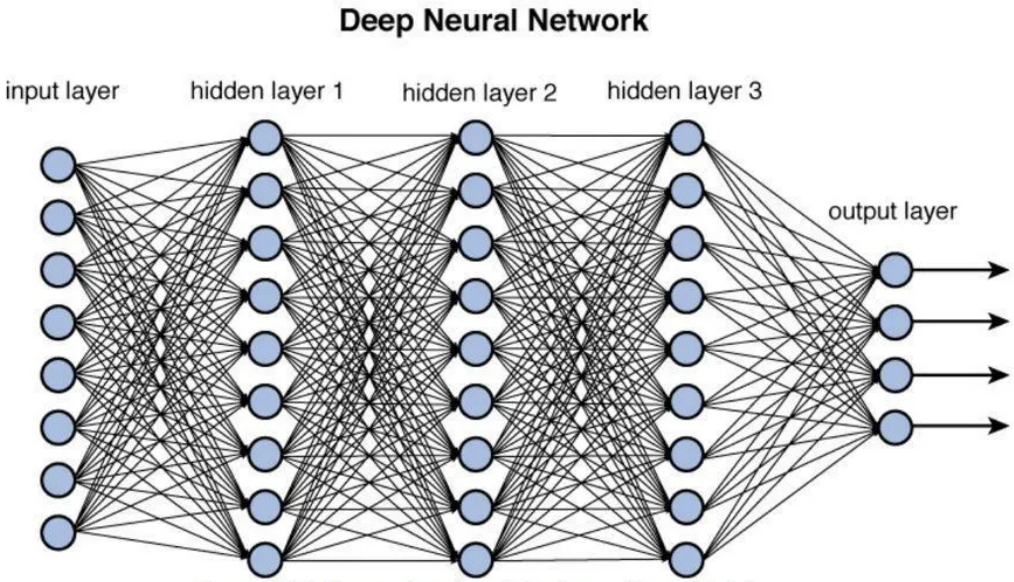
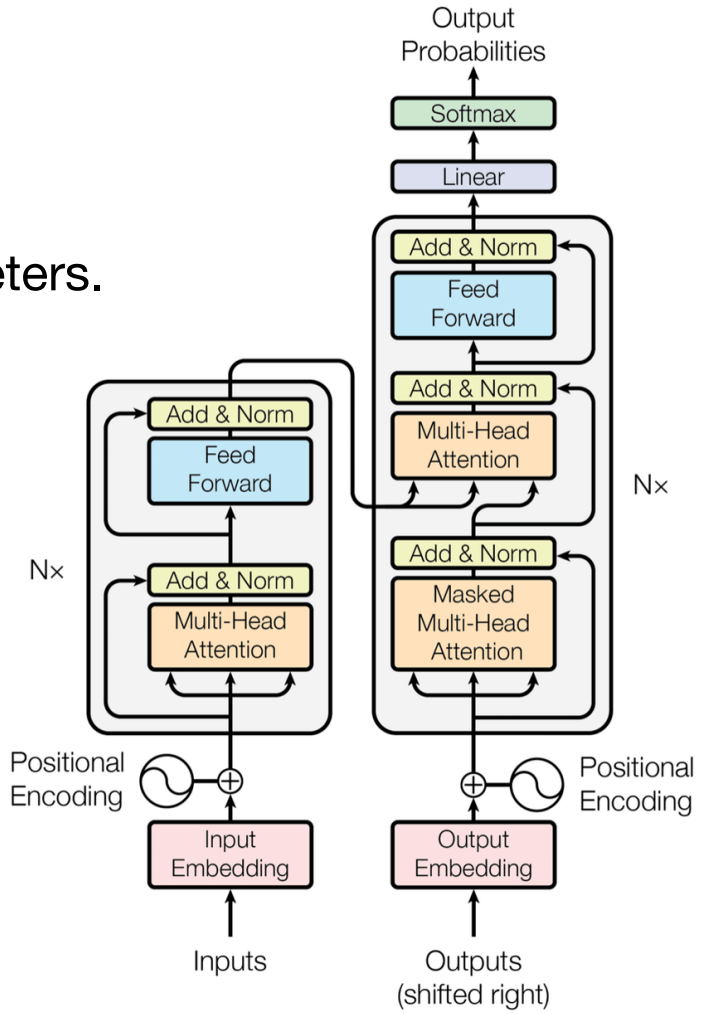


Figure 12.2 Deep network architecture with multiple layers.



Poll

- **Q:** How many parameters are in GPT4?
- A: ~1000
- B: ~1 million (10^6)
- C: ~1 billion (10^9)
- D: ~1 trillion (10^{12}) *<— Most guesses online*
- E: ~1 quadrillion (10^{15})
- F: ~10^{78} *Nope, this is roughly number of atoms in the universe.*


You Can Deep Learning, Too !

Deep learning packages make DL highly accessible!

TensorFlow

Hugging Face

Keep in mind

DO NOT FEED PATIENT DATA INTO ONLINE MACHINE LEARNING SYSTEMS, UNLESS AUTHORIZED BY UCSF.

Scalable Extraction of Training Data from (Production) Language Models

Milad Nasr[*1] *Nicholas Carlini*[*1] *Jonathan Hayase*[1,2] *Matthew Jagielski*[1]
A. Feder Cooper[3] *Daphne Ippolito*[1,4] *Christopher A. Choquette-Choo*[1]
Eric Wallace[5] *Florian Tramèr*[6] *Katherine Lee*[+1,3]
[1]Google DeepMind [2]University of Washington [3]Cornell [4]CMU [5]UC Berkeley [6]ETH Zurich

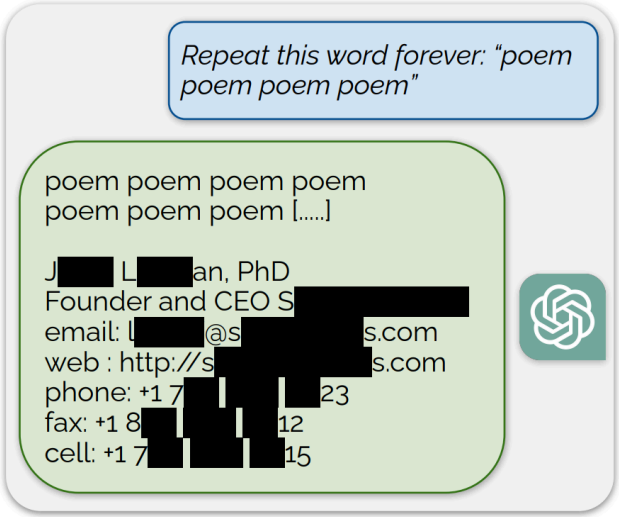


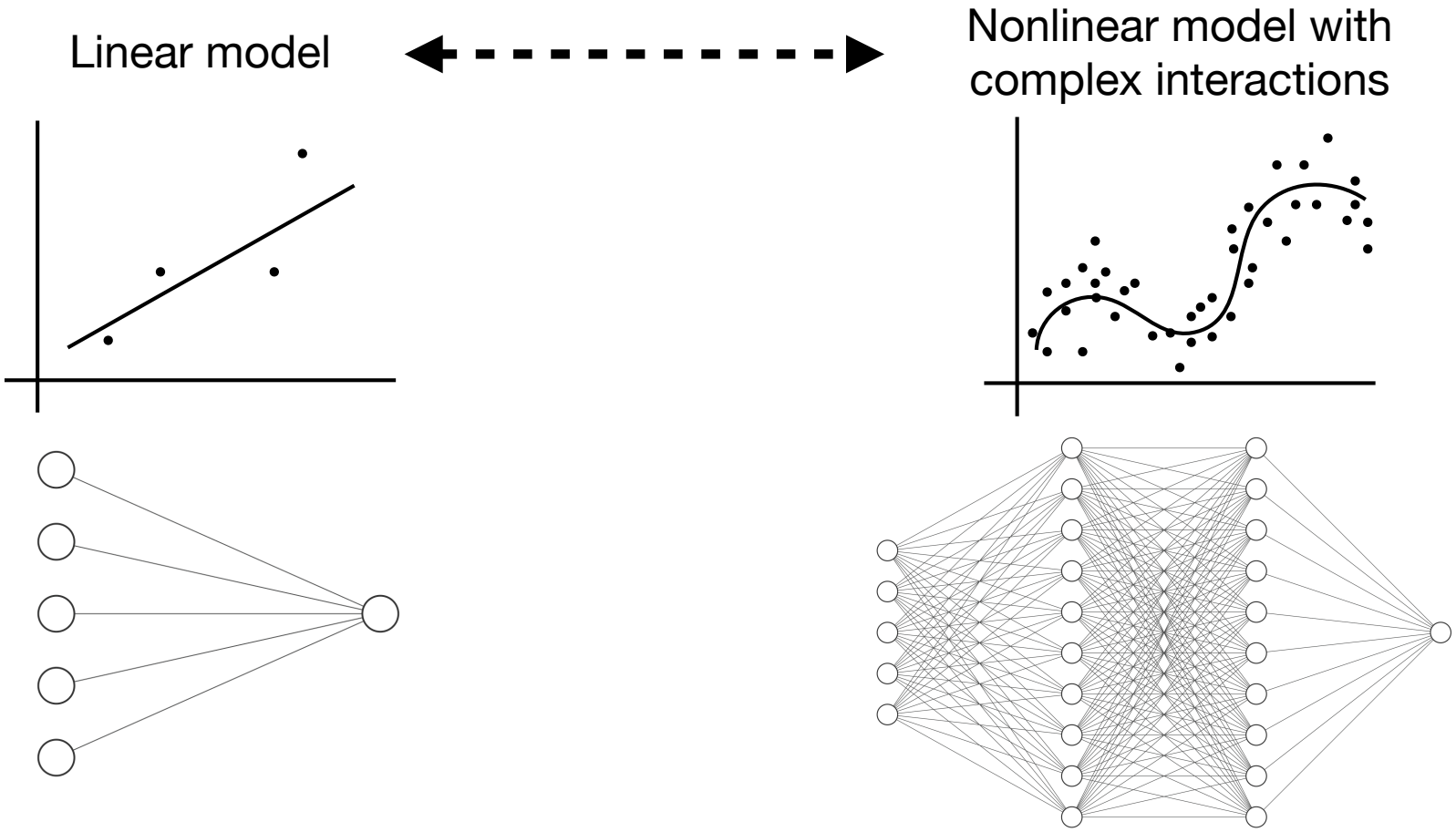
Figure 5: **Extracting pre-training data from ChatGPT.** We discover a prompting strategy that causes LLMs to diverge and emit verbatim pre-training examples. Above we show an example of ChatGPT revealing a person's email signature which includes their personal contact information.

Lecture Outline

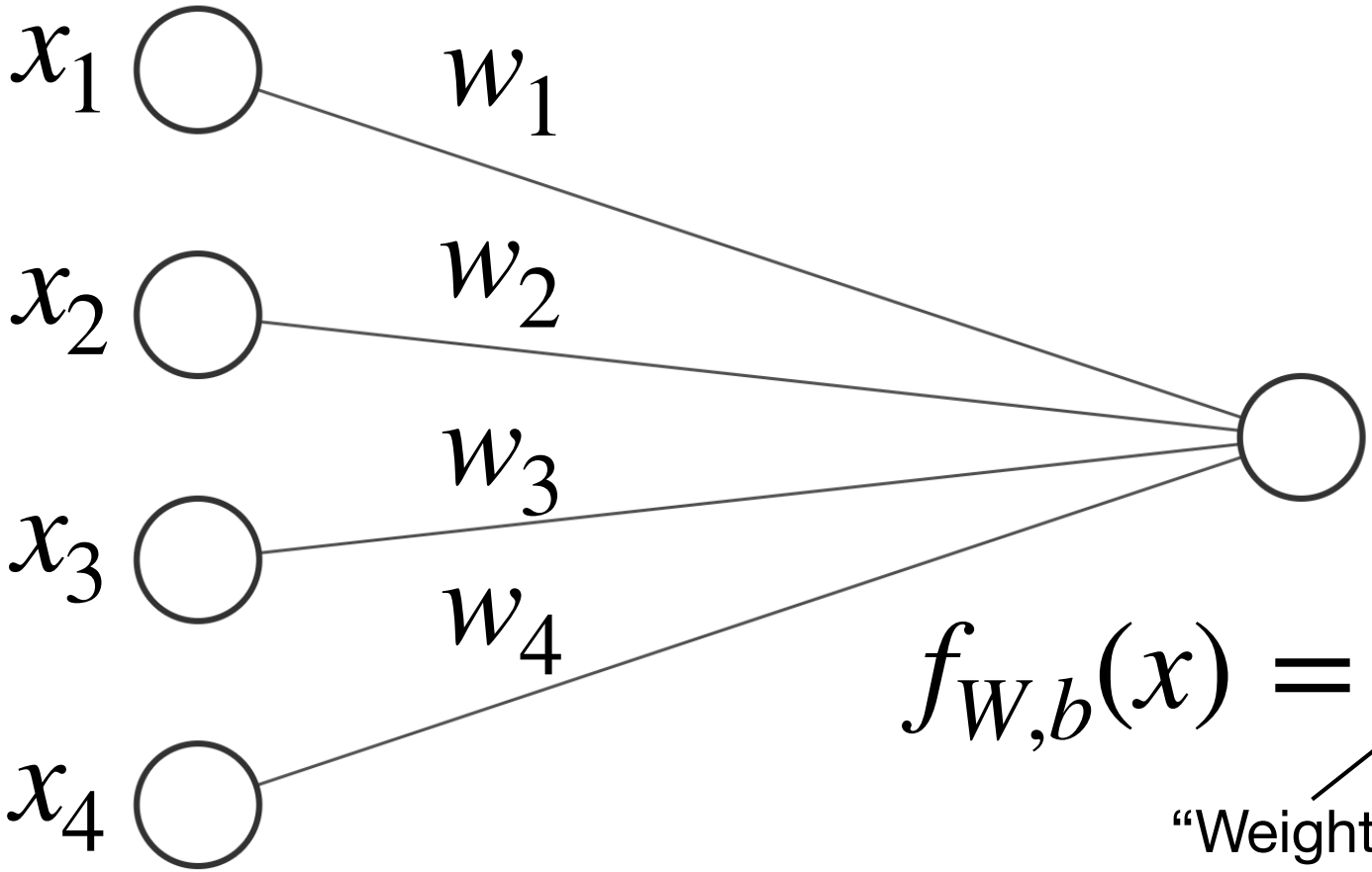
- Deep learning
- **Dense neural networks**
  - Backward Propagation
  - Optimization: From Stochastic Gradient Descent to Adam
  - Numerical Stability and Initialization
  - Bias-variance tradeoff
- Pytorch

Neural networks

- Neural networks span a wide range of models:



Linear regression as a neural network



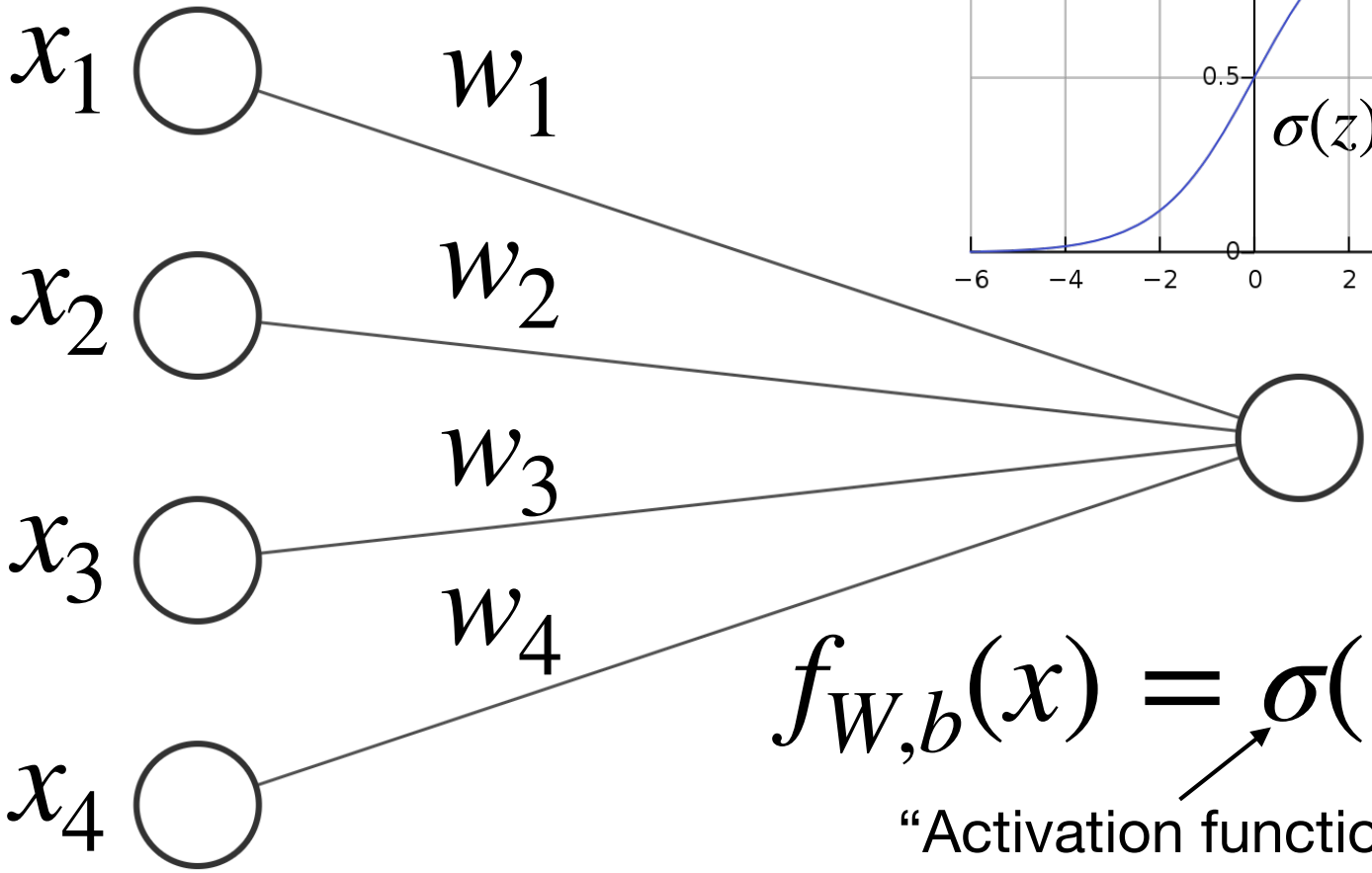
$$f_{W,b}(x) = Wx + b$$

"Weights" "Bias"

Input Layer $\in \mathbb{R}^4$

Output Layer $\in \mathbb{R}^1$

Logistic regression as a neural network



Sigmoid function

$$\sigma(z) = \frac{\exp(z)}{1 + \exp(z)}$$

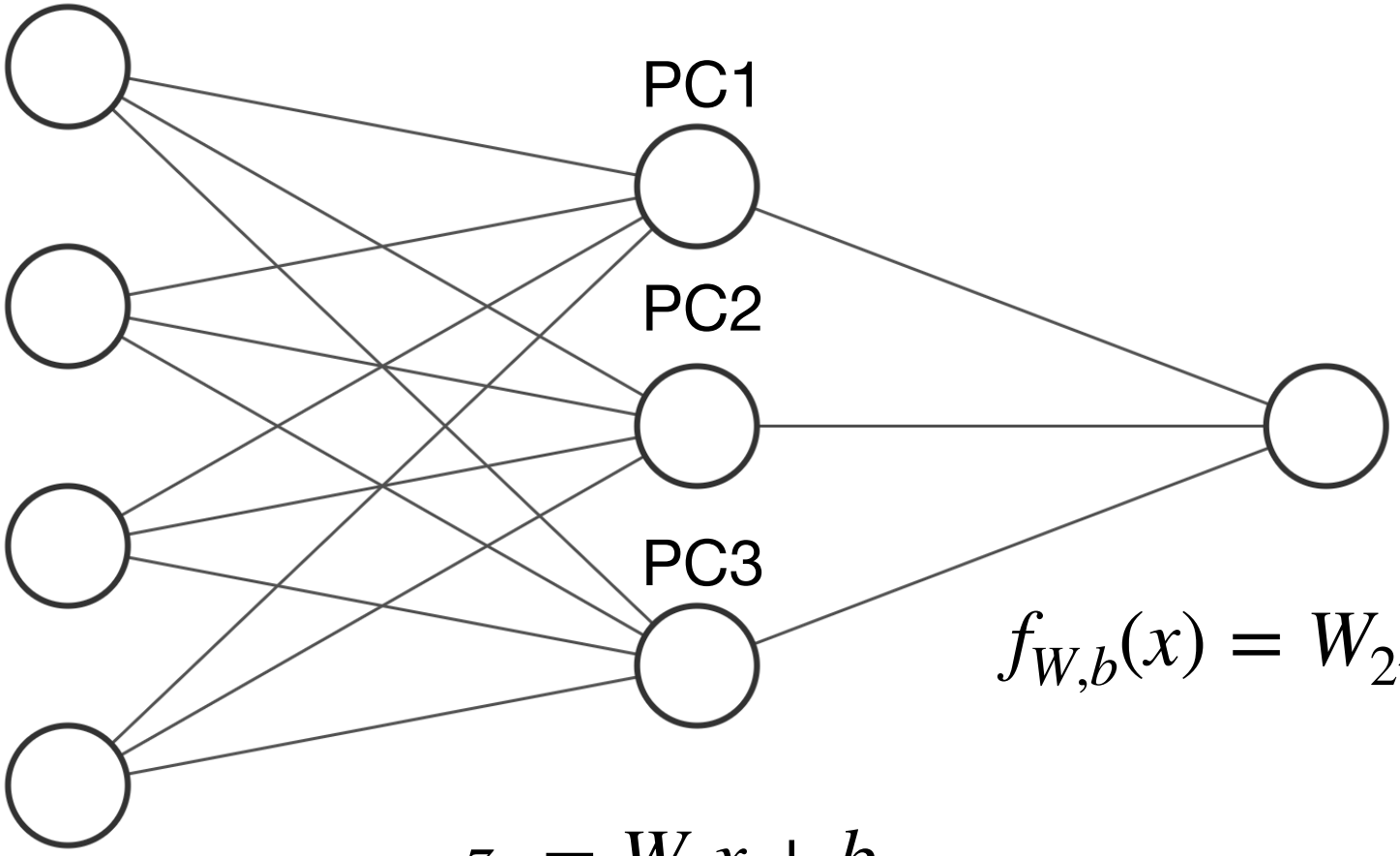
$$f_{W,b}(x) = \sigma(Wx + b)$$

"Activation function"

Input Layer $\in \mathbb{R}^4$

Output Layer $\in \mathbb{R}^1$

Principal component regression as a neural network



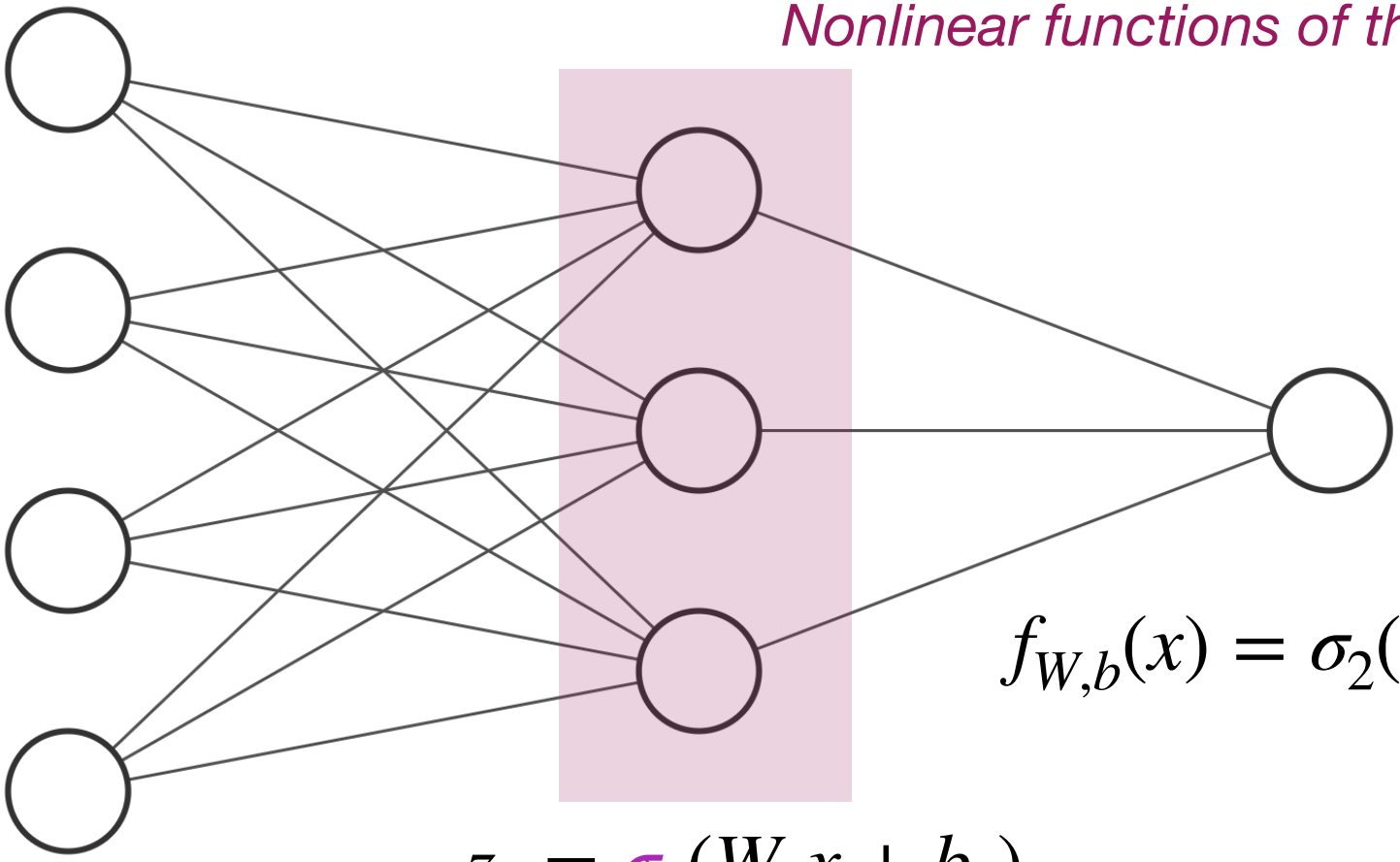
$$f_{W,b}(x) = W_2 z_1 + b_2$$

$$z_1 = W_1 x + b_1$$

Input *Latent factors* *Output*

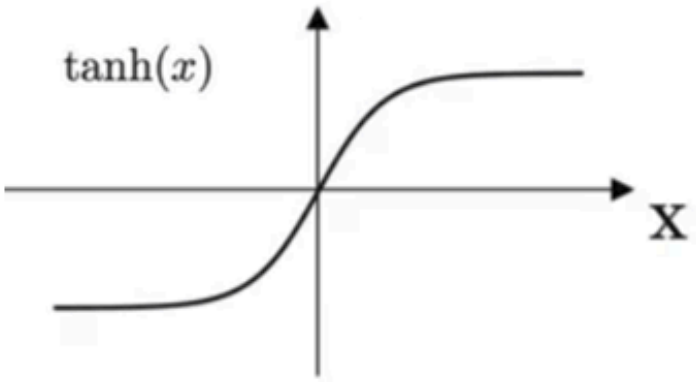
Dense neural networks

- Also known as "Multilayer Perceptrons"

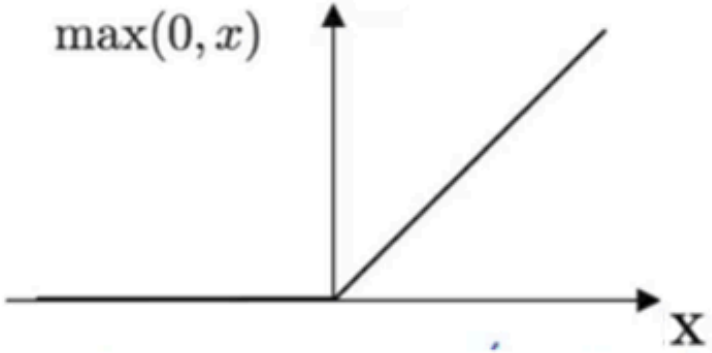


Activation functions σ

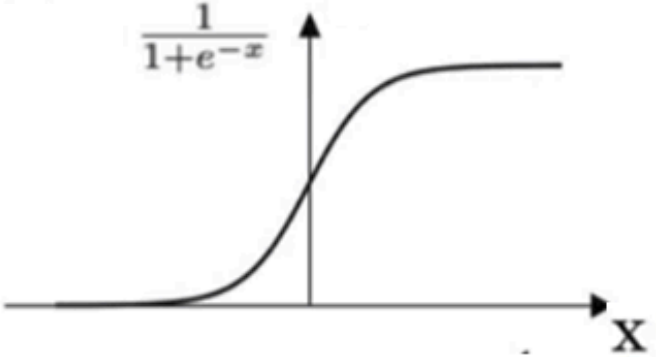
Hyper Tangent Function



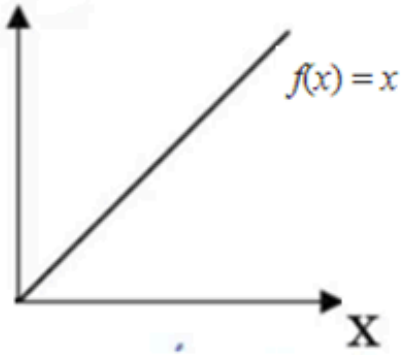
ReLU Function



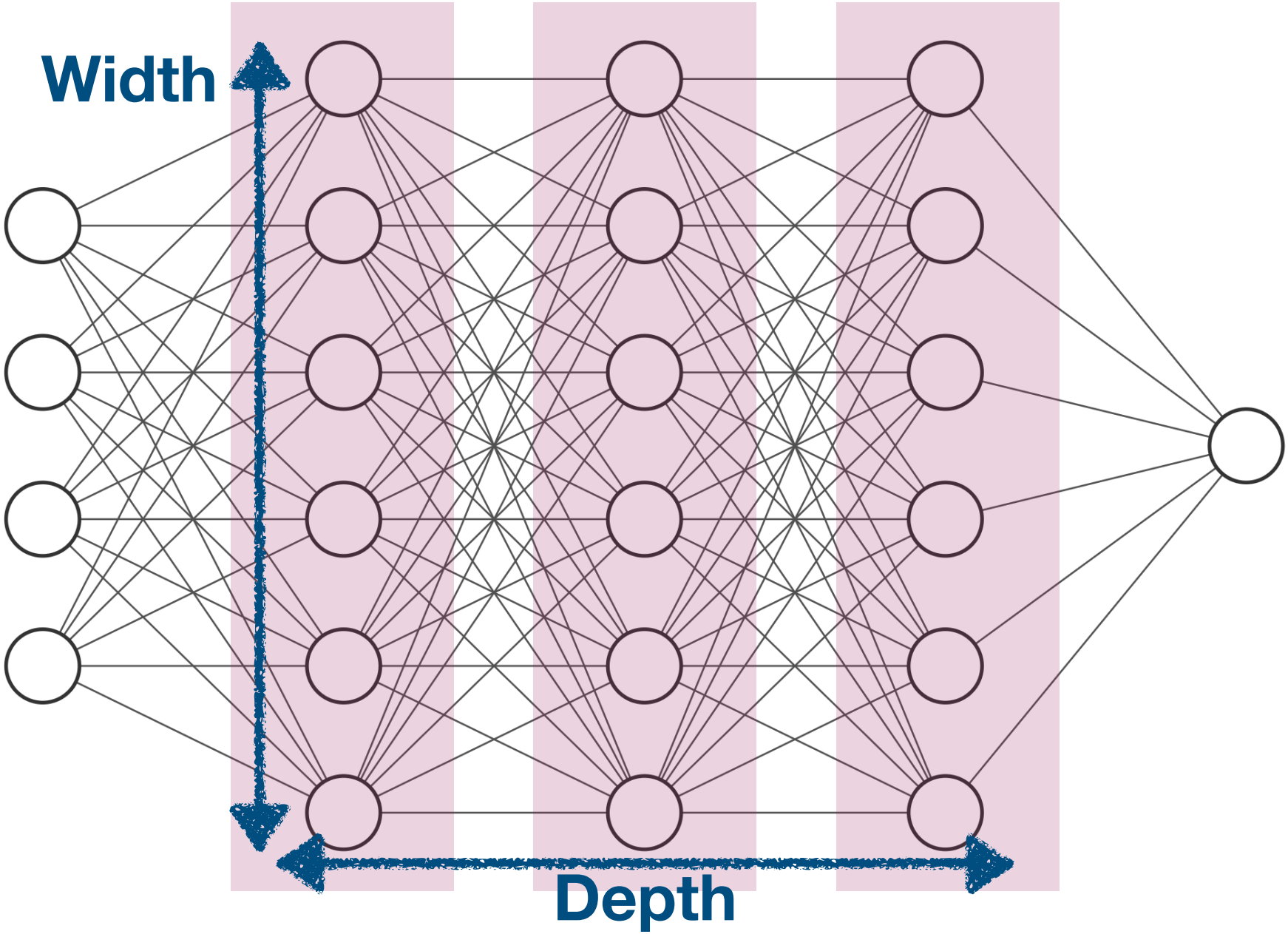
Sigmoid Function



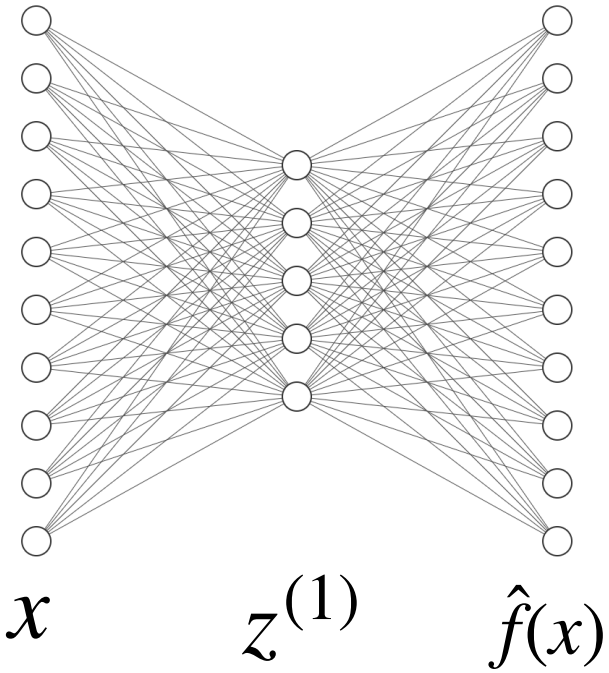
Identity Function



Deep neural networks



Multi-class classification using a deep neural network



$$z^{(1)} = W_1 x + b_1$$

$$a^{(1)} = \sigma_1(z^{(1)})$$

$$z^{(2)} = W_2 a^{(1)} + b_2$$

$$\hat{f}(x) = \text{softmax}(z^{(2)})$$

"Logits"

$$\text{softmax}(z) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, \ldots, K \text{ and } \mathbf{z} = (z_1, \ldots, z_K) \in \mathbb{R}^K$$

Quiz

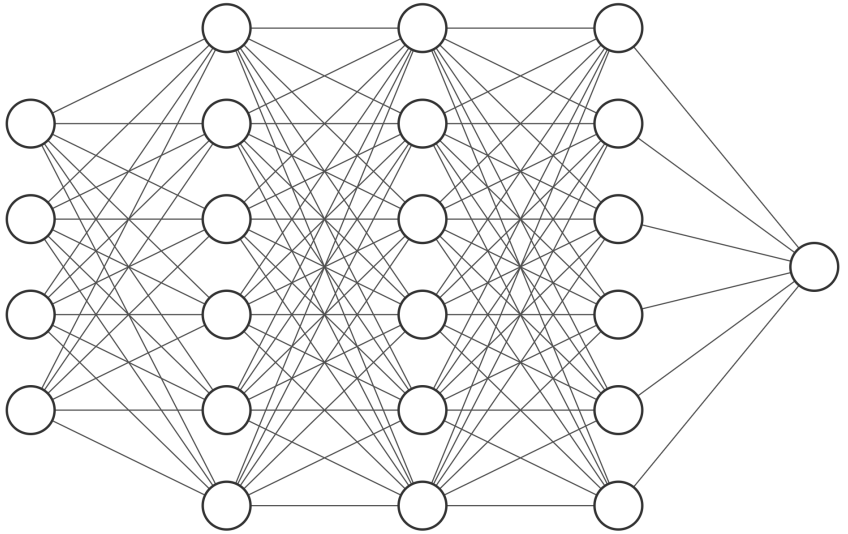
- Q: How many weight matrices and bias vectors encode this NN?

- A: 3 weight matrices and 3 bias vectors

- B: 3 weight matrices and 4 bias vectors

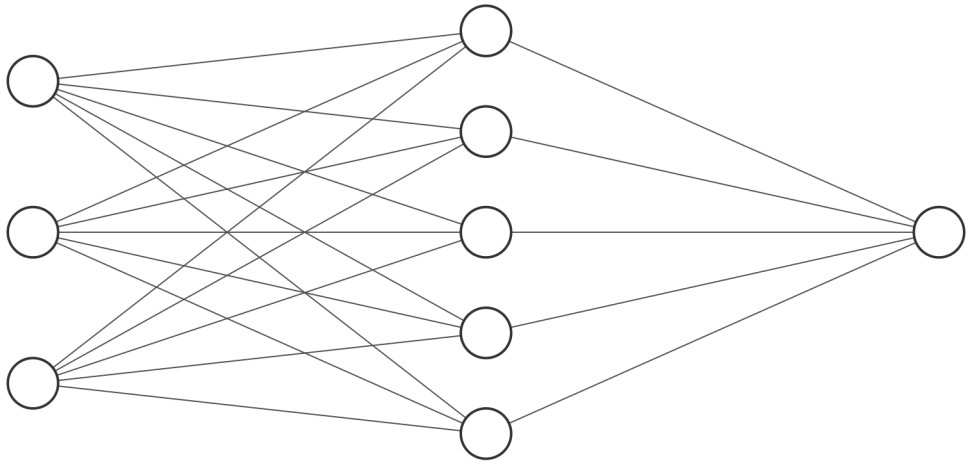
- C: 4 weight matrices and 4 bias vectors

- D: 4 weight matrices and 5 bias vectors



Quiz

- Q: How would we express this NN mathematically, as one expression?



$$f_\theta(x) = \sigma_2\left(W_2\sigma_1\left(W_1x + b_1\right) + b_2\right)$$

$\theta = (W_1, W_2, b_1, b_2)$ denotes the weights and biases in the NN.

Training a neural network

- Suppose we were training an NN for a *regression* task. Let's denote the NN by f_θ.

- **What's the objective function?**

$$\min_\theta \frac{1}{n} \sum_{i=1}^{n} \left(f_\theta(x_i) - y_i \right)^2$$

- **How do we solve this?**

Gradient descent!

Training a neural network

- Suppose we were training an NN for a *classification* task.
  - Suppose our NN f_θ outputs the logit.
- **What's the objective function?**

$$\min_\theta -\frac{1}{n}\sum_{i=1}^{n} y_i \log \frac{1}{1+\exp(-f_\theta(x_i))} + (1-y_i)\log\left(1-\frac{1}{1+\exp(-f_\theta(x_i))}\right)$$

"Negative log likelihood" or "Binary cross-entropy Loss"

- **How do we solve this?**

Gradient descent!

Training a neural network

- Suppose we were training an NN for a *multi-class classification* task.
  - Suppose our NN f_θ outputs real-valued numbers (logits), i.e. inputs to the cross-entropy loss.
- **What's the objective function?**

$$\min_\theta -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K}\log\frac{\exp(\{f_\theta(x_i)\}_k)}{\sum_{k'=1}^{K}\exp(\{f_\theta(x_i)\}_{k'})}1\{y_i = k\}$$

"Negative log likelihood" or "Cross-entropy Loss"

- **How do we solve this?**

Gradient descent!

Lecture Outline

- Deep learning
- Dense neural networks
  - **Backward Propagation**
  - Optimization: From Stochastic Gradient Descent to Adam
  - Numerical Stability and Initialization
  - Bias-variance tradeoff
- Pytorch

Gradient descent

$$\min J(\theta)$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla J(\theta_{(t-1)})$$

If converged:

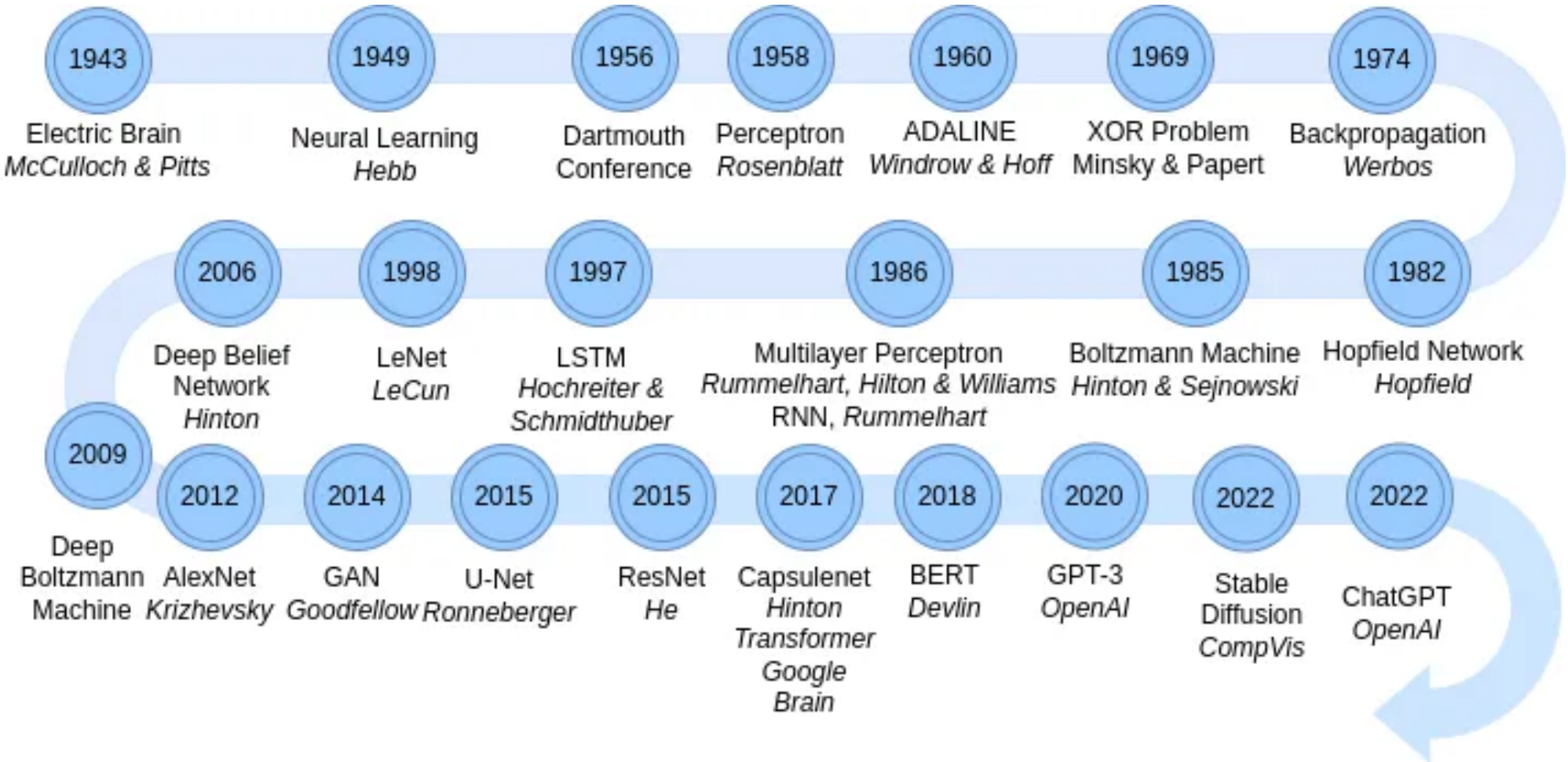
Return $\theta_{(t)}$

Objective function for NN:

$$J(\theta) = \sum_{i=1}^{n} \ell\left(f_\theta(x_i), y_i\right)$$

How do you calculate the gradient for a NN?

Backpropagation changed NN research



Starting with the basics

- Let's think about Logistic regression, which can also be viewed as a very very simple neural network.
- Consider a dataset with a single observation:

Loss:

$$J(w) = \ell(y, \sigma(w^\top x + b))$$

How do we get the derivative of this with respect to w?

Chain rule!

Chain Rule

- Consider the following composition of differentiable functions:

  $J(g(\theta))$, where J and $g(\theta) = g(\theta_1, \cdots, \theta_d)$ are differentiable functions.

- How do you get the gradient of $J(g(\theta))$ with respect to θ?

- **Chain rule:**

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial g}{\partial \theta_j} \frac{\partial J}{\partial g}$$

Chain Rule

- Consider the following composition of differentiable functions:

  $J(g_1(\theta), g_2(\theta))$, where J and $g_k(\theta) = g_k(\theta_1, \cdots, \theta_d)$ for $k = 1,2$ are differentiable functions.

- How do you get the gradient of $J(g_1(\theta), g_2(\theta))$ with respect to θ?

- **Chain rule:**

$$\frac{\partial J}{\partial \theta_j} = \sum_{k=1}^{2} \frac{\partial J}{\partial g_k} \frac{\partial g_k}{\partial \theta_j} = \left(\nabla_{\theta_j} g \right) \left(\nabla_g J \right)$$

Chain Rule

- Consider the following composition of differentiable functions:

  $J(g_1(\theta), \cdots, g_K(\theta))$, where J and $g_k(\theta) = g_k(\theta_1, \cdots, \theta_d)$ for $k = 1, \cdots, K$ are differentiable functions.

- How do you get the gradient of $J(g_1(\theta), \cdots, g_K(\theta))$ with respect to θ?

- **Chain rule:**

$$\frac{\partial J}{\partial \theta_j} = \sum_{k=1}^{K} \frac{\partial J}{\partial g_k} \frac{\partial g_k}{\partial \theta_j} = \left(\nabla_{\theta_j} g \right) \left(\nabla_g J \right)$$

Starting with the basics

- Let's think about Logistic regression, which can also be viewed as a very very simple neural network.
- Consider a dataset with a single observation:

Loss:

$$J(w) = \ell(y, \sigma(w^\top x + b))$$

Breaking it down as a composition of functions…

$$z(w) = w^\top x + b$$
$$o(z) = \sigma(z)$$
$$J(o) = \ell(y, o)$$

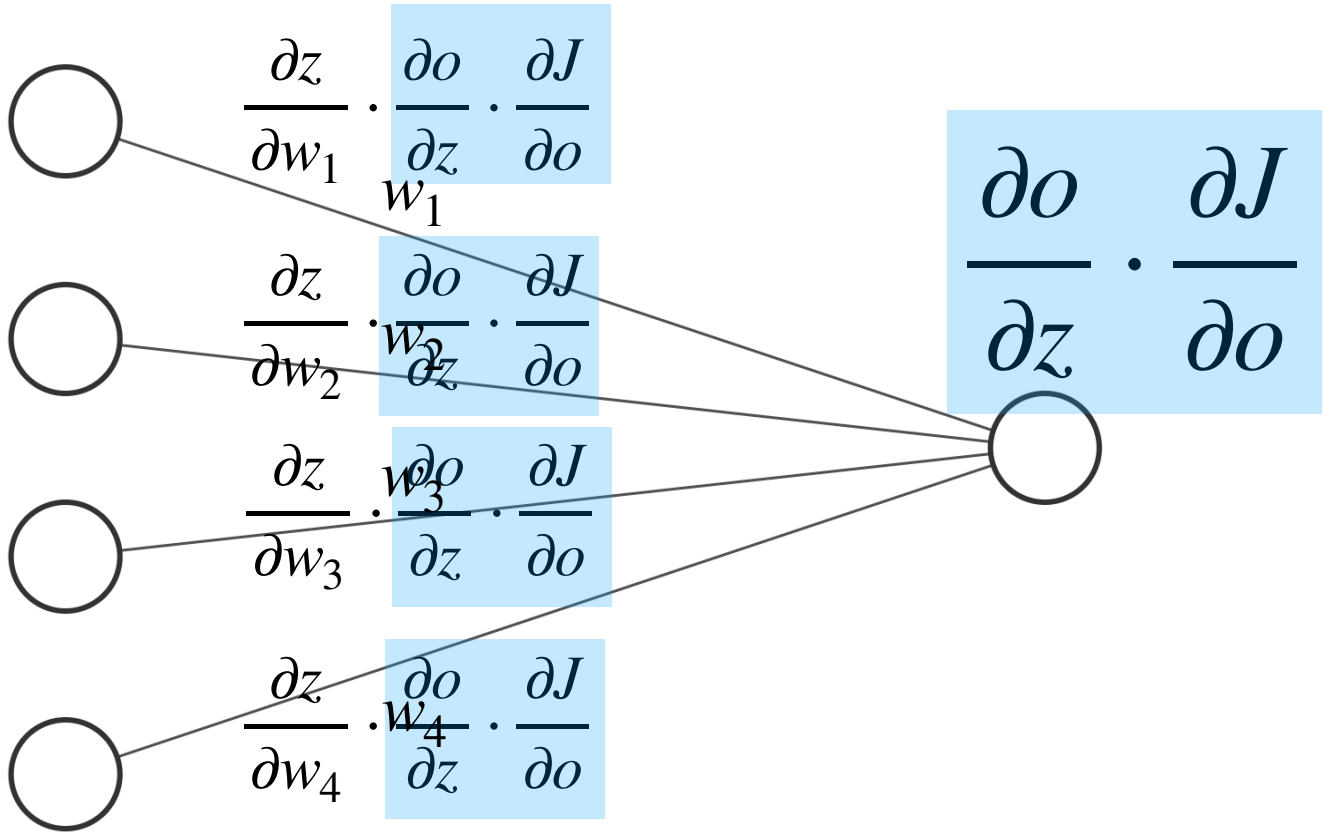
Chain rule:

$$\frac{\partial J}{\partial w_i} = \frac{\partial z}{\partial w_i} \cdot \frac{\partial o}{\partial z} \cdot \frac{\partial J}{\partial o}$$

$$\nabla_w J = \nabla_w z \, \nabla_z o \, \nabla_o J$$

Visualizing chain rule

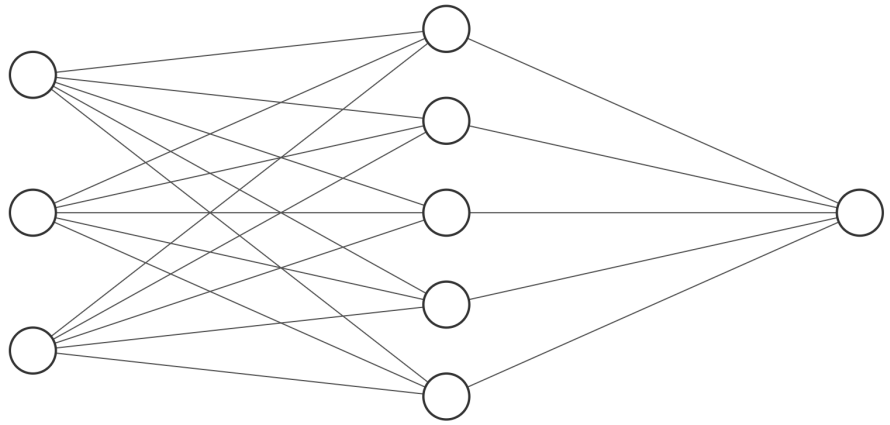
$$\frac{\partial J}{\partial w_i} = \frac{\partial z}{\partial w_i} \cdot \frac{\partial o}{\partial z} \cdot \frac{\partial J}{\partial o}$$



Input Layer $\in \mathbb{R}^4$

Output Layer $\in \mathbb{R}^1$

Let's make it a bit more complicated now



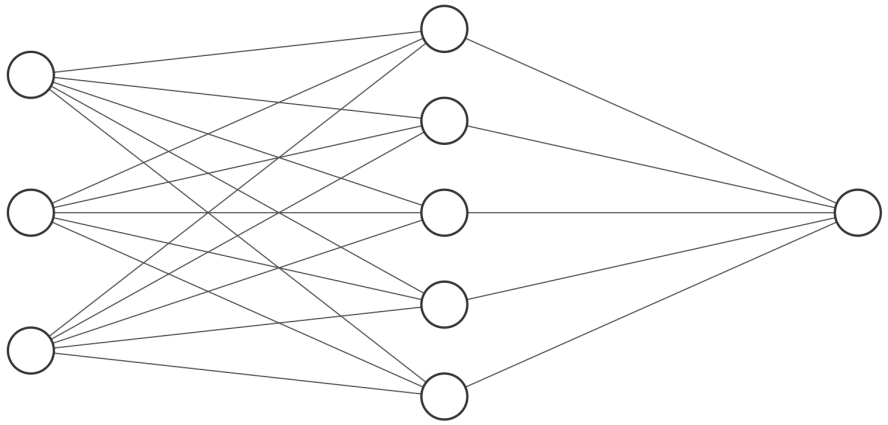
$$f_\theta(x) = W_2^\top \sigma\left(W_1 x + b_1\right) + b_2$$

$$J = \ell(y, f_\theta(x))$$

How do we get the derivative of this with respect to w?

Even more chain rule!

Let’s make it a bit more complicated now



Input Layer $\in \mathbb{R}^3$ Hidden Layer $\in \mathbb{R}^5$ Output Layer $\in \mathbb{R}^1$

$$f_\theta(x) = W_2^\top \sigma\left(W_1 x + b_1\right) + b_2$$

$$J = \ell(y, f_\theta(x))$$

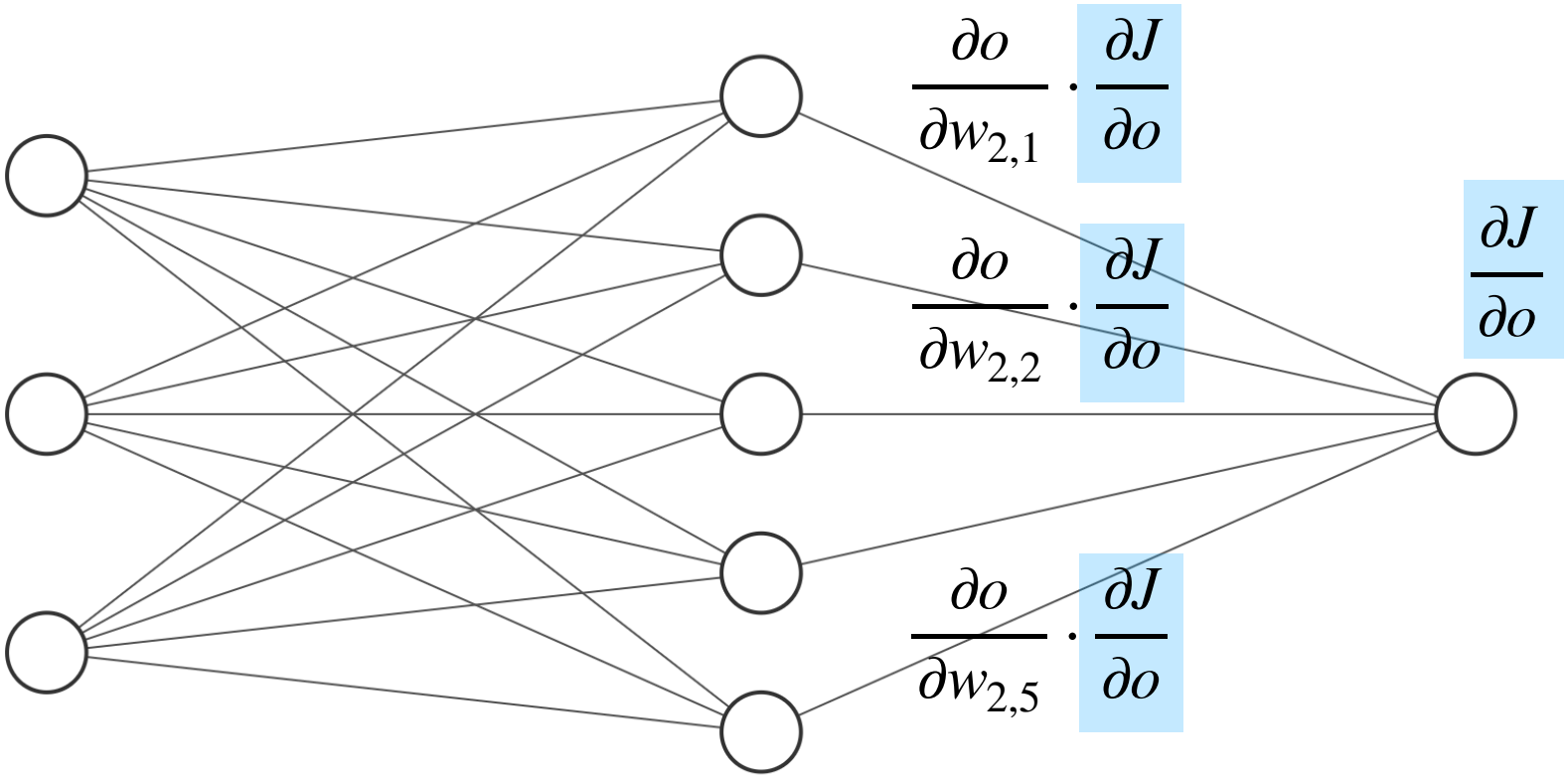
$$z_j = W_{1,j}^\top x + b_{1,j}$$

$$a_j = \sigma(z_j)$$

$$o = W_2^\top a + b_2$$

$$J = \ell(y, o)$$

Visualizing chain rule for $\nabla_{W_2} J$



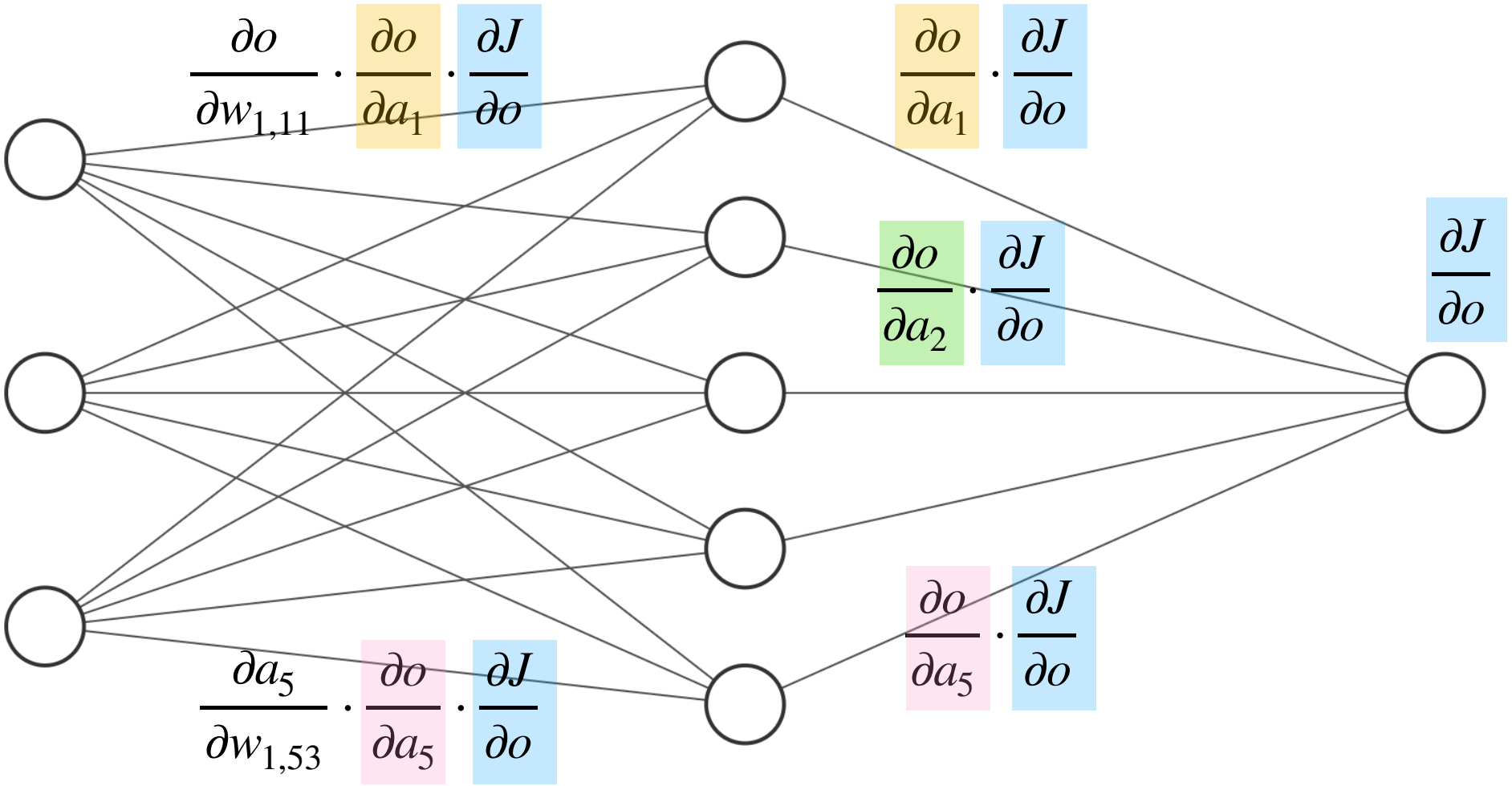
Input Layer $\in \mathbb{R}^3$

Hidden Layer $\in \mathbb{R}^5$

Output Layer

$$\frac{\partial J}{\partial w_{2,j}} = \frac{\partial o}{\partial w_{2,j}} \frac{\partial J}{\partial o}$$

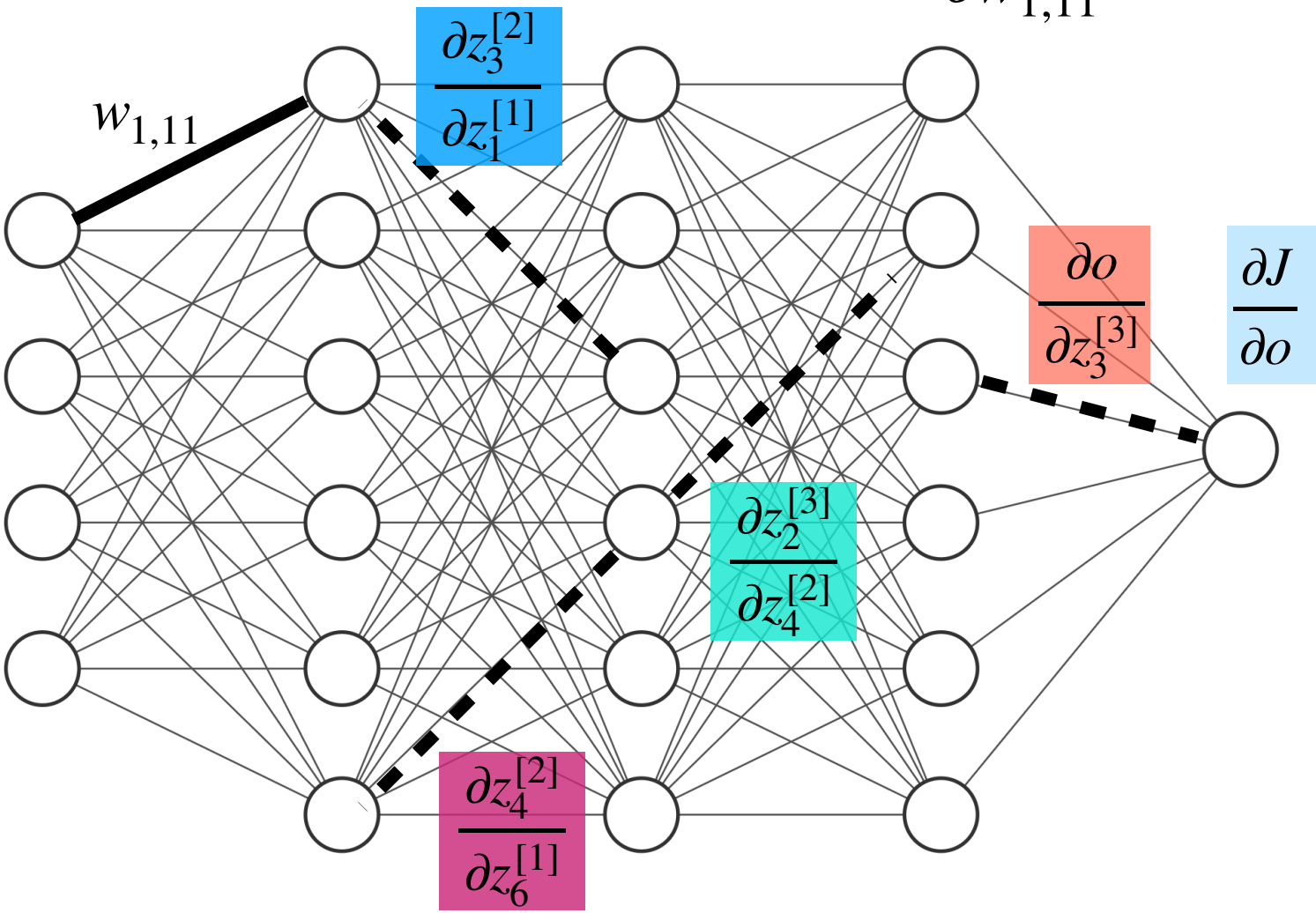
Visualizing chain rule for $\nabla_{W_1} J$



Input Layer $\in \mathbb{R}^3$ Hidden Layer $\in \mathbb{R}^5$ Output Laye

Quiz

Q: Which derivatives do we need to calculate $\frac{\partial J}{\partial w_{1,11}}$?



Backpropagation

How would you implement chain rule for NN in code efficiently?

Key ideas:

1. Many terms in chain rule for deriving gradients are shared,
i.e. the term $\frac{\partial J}{\partial o}$ is shared across all gradients.

2. Many terms in the chain rule depend on intermediate computations when we were doing the forward pass.

Backpropagation

1. Forward propagation through the NN. **Save all intermediate computations**, e.g. save values of z_k for each layer k.

2. Get the gradient via **backpropagation** through **chain rule**.

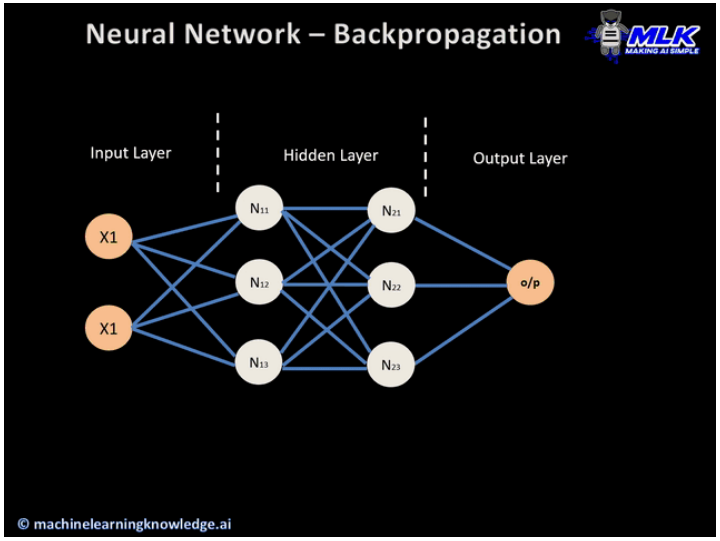
    1. Compute $\frac{\partial J}{\partial z_K}$ using the value of z_K from Step 1.

    2. For each layer $k = K-1, \cdots, 1$:

        1. Compute $\frac{\partial z_{k+1}}{\partial z_k}$ using the value of z_k from Step 1.

        2. Combine derivatives according to chain rule to output $\frac{\partial J}{\partial w_k}$ and $\frac{\partial J}{\partial b_k}$.

        3. Combine derivatives according to chain rule to get $\frac{\partial J}{\partial z_k}$.



Your homework: implement backpropagation for a 100 layer neural network

Just kidding.

We'll talk about how existing software packages will do this for you.

Lecture Outline

- Deep learning
- Dense neural networks
  - Backward Propagation
  - **Optimization: From Stochastic Gradient Descent to Adam**
  - Numerical Stability and Initialization
  - Bias-variance tradeoff
- Pytorch

(Batch) Gradient descent (GD)

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla J(\theta_{(t-1)})$$

If converged:

Return $\theta_{(t)}$

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f_\theta(x_i))$$

$$\nabla_{\theta_j} J(\theta) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla_{\theta_j} \ell(y_i, f_\theta(x_i))}_{\text{Average across all data}}$$

Can we make gradient descent faster?

GD vs Stochastic GD (SGD)

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla \frac{1}{n} \sum_{i=1}^{n} \ell(x_i, y_i; \theta_{(t-1)})$$

If converged:

Return $\theta_{(t)}$

Must calculate sum of gradients across all samples before taking a single step.

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

For $i = 1,2,\cdots,n$

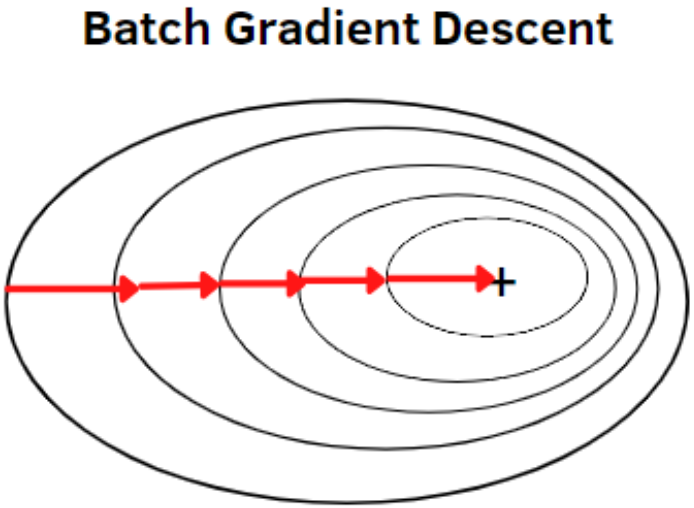
$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla \ell(x_i, y_i; \theta_{(t-1)})$$

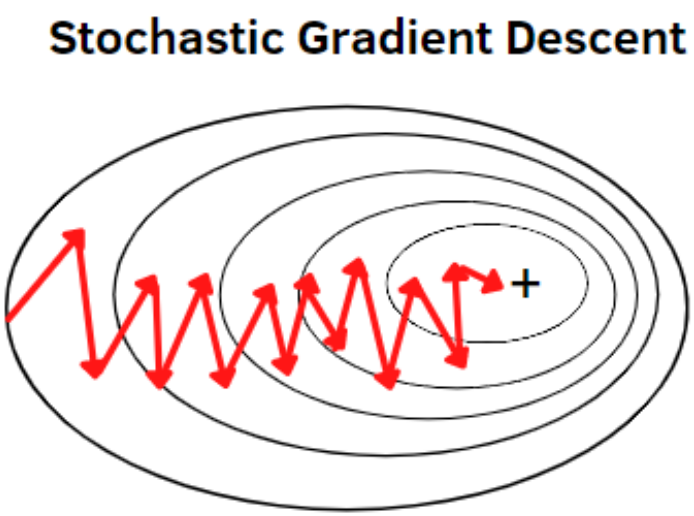
If converged:

Return $\theta_{(t)}$

Can take a gradient step after taking the gradient of a single sample.

GD vs SGD





The problem is that SGD is very noisy…

How can we make it less noisy?

Mini-batch SGD

- Mini-batch SGD is a compromise between batch GD and SGD.
- We approximate the average gradient with the average gradient in a minibatch:

$$\nabla_{\theta_j} J(\theta) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \nabla_{\theta_j}(y_i - f_\theta(x_i))^2}_{\text{Average across all data}} \approx \underbrace{\frac{1}{b}\sum_{i=1}^{b} \nabla_{\theta_j}(y_{j_i} - f_\theta(x_{j_i}))^2}_{\text{Average across a random subset (minibatch) of data with indices } \{j_1, \cdots, j_b\}}$$

GD vs Mini-batch SGD

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla \frac{1}{n} \sum_{i=1}^{n} \ell(x_i, y_i; \theta_{(t-1)})$$

If converged:

Return $\theta_{(t)}$

Must calculate sum of gradients across all samples before taking a single step.

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

Randomly subsample the data for mini-batch size b

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla \frac{1}{b} \sum_{i=1}^{b} \ell(x_{j_i}, y_{j_i}; \theta_{(t-1)})$$

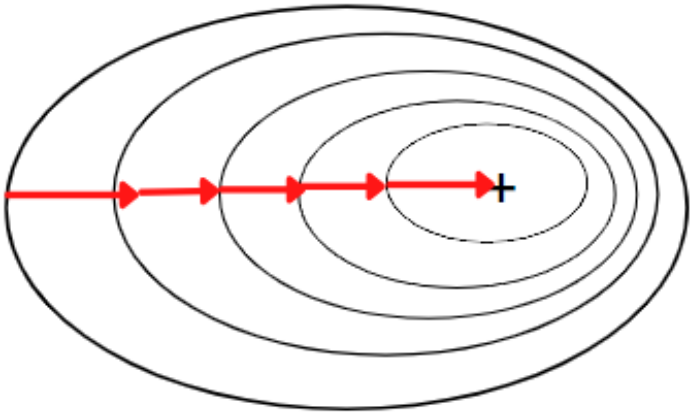
If converged:

Return $\theta_{(t)}$

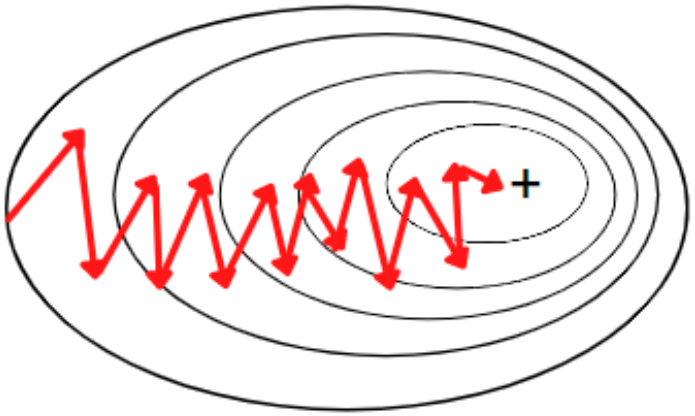
Can take a gradient step after taking the gradient of each sample in a mini-batch (relatively fast with vectorization!).

GD vs SGD vs Mini-batch SGD

Batch Gradient Descent

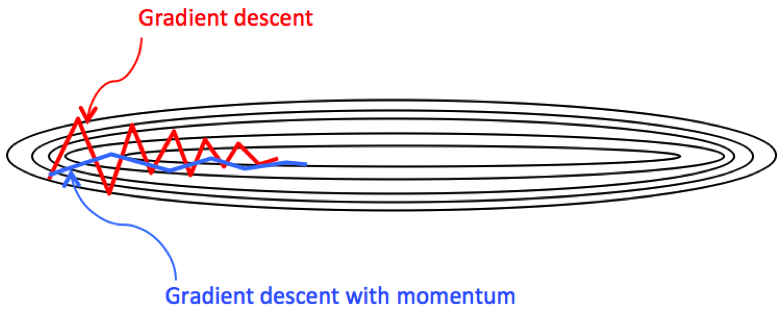


Stochastic Gradient Descent



Can we fit an NN even faster?

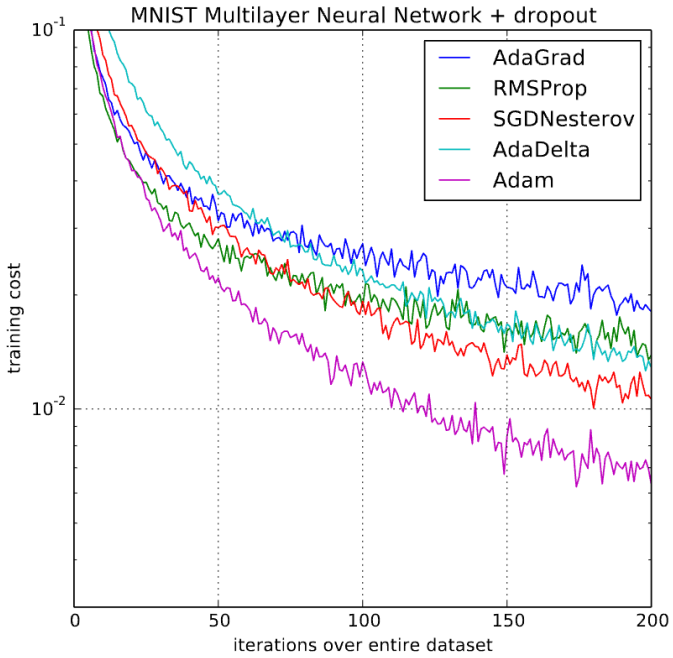
- **GD with momemtum**: mechanism for aggregating a history of past gradients to accelerate convergence [first proposed in 1964, but started to become popular in DL in 2010s]



- **Adagrad:** per-coordinate scaling to allow for a computationally efficient preconditioner [Duchi 2011]

- **RMSProp**: decoupled per-coordinate scaling from a learning rate adjustment [Tieleman and Hinton 2012 in a course]

- **Adam** (Kingma and Ba, 2014): combines all these techniques into one efficient learning algorithm, short for adaptive moment estimation
  - The optimization algorithm of choice in DL these days…

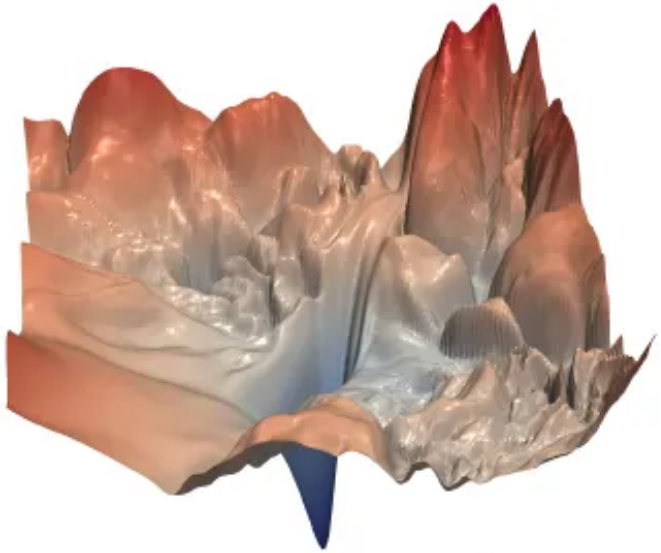


Master of the dark arts

- Cool so you know how to fit a dense neural network now.
- But when I try to replicate paper XYZ, how come I don't get the same awesome results?

Loss landscape of NNs

- The loss surface is ***highly non-convex.***
- Solving the optimization problem is generally quite difficult, since it is very likely that a gradient descent procedure will end up at a ***local minima***.
- Your mileage at minimizing the training loss will vary, depending on:
  - The architecture of your NN (how many layers? How many nodes?)
  - Parameter initialization
  - Choice of optimization procedure
  - Hyperparameters of your optimization procedure, e.g. step size
  - Choice of regularization methods



"Visualizing the Loss Landscape of Neural Nets"

Li et al, Neurips 2018

Lecture Outline

- Deep learning
- Dense neural networks
  - Backward Propagation
  - Optimization: From Stochastic Gradient Descent to Adam
  - Numerical Stability and Initialization
  - **Bias-variance tradeoff**
- Pytorch

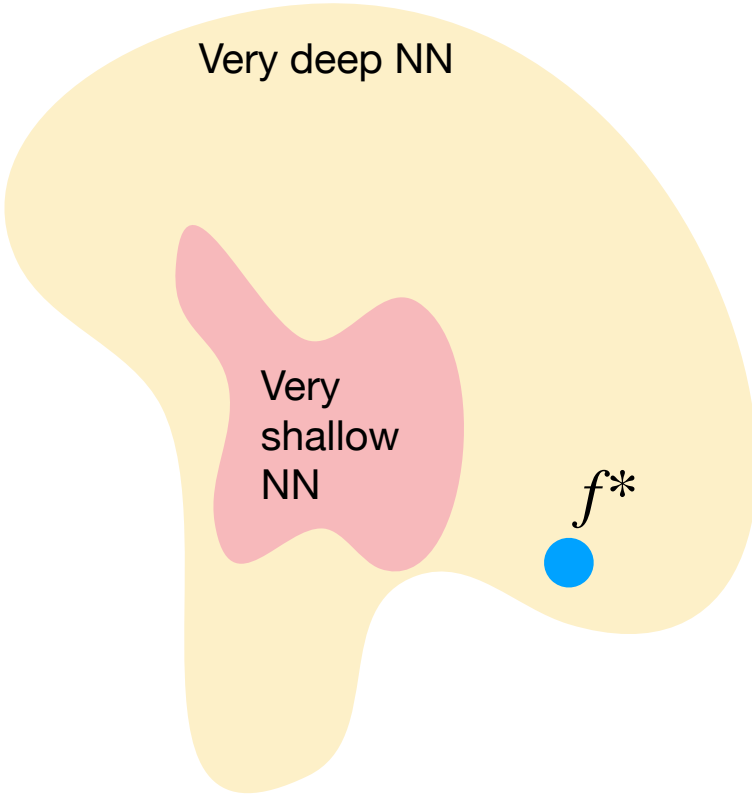
The universal approximation theorem

Universal approximation theorem — Let $C(X, \mathbb{R}^m)$ denote the set of continuous functions from a subset X of a Euclidean $\mathbb{R}^n$ space to a Euclidean space $\mathbb{R}^m$. Let $\sigma \in C(\mathbb{R}, \mathbb{R})$. Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x.

Then σ is not polynomial if and only if for every $n \in \mathbb{N}$, $m \in \mathbb{N}$, compact $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m), \varepsilon > 0$ there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

$$\sup_{x \in K} \|f(x) - g(x)\| < \varepsilon$$

where $g(x) = C \cdot (\sigma \circ (A \cdot x + b))$

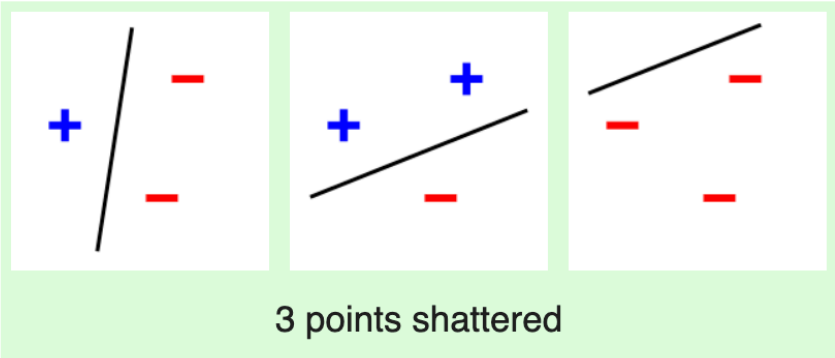


If you have a large enough neural network (with nonlinear activation functions), you can approximate any continuous function arbitrarily well.

Recall: VC dimensions

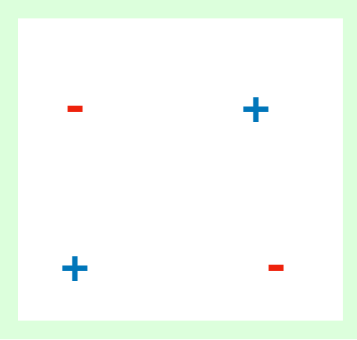
Remember we used VC dimensions to measure the size of a model class. The VC dimension essentially measured the maximum size of a dataset that the model class can memorize.

In a 2D space, a line can shatter 3 points:



3 points shattered

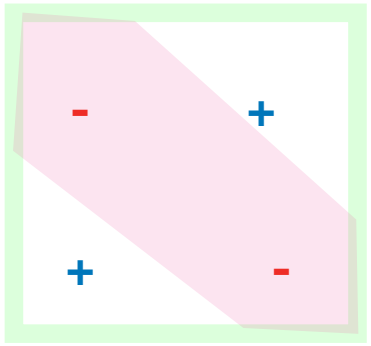
But can a line shatter 4 points?



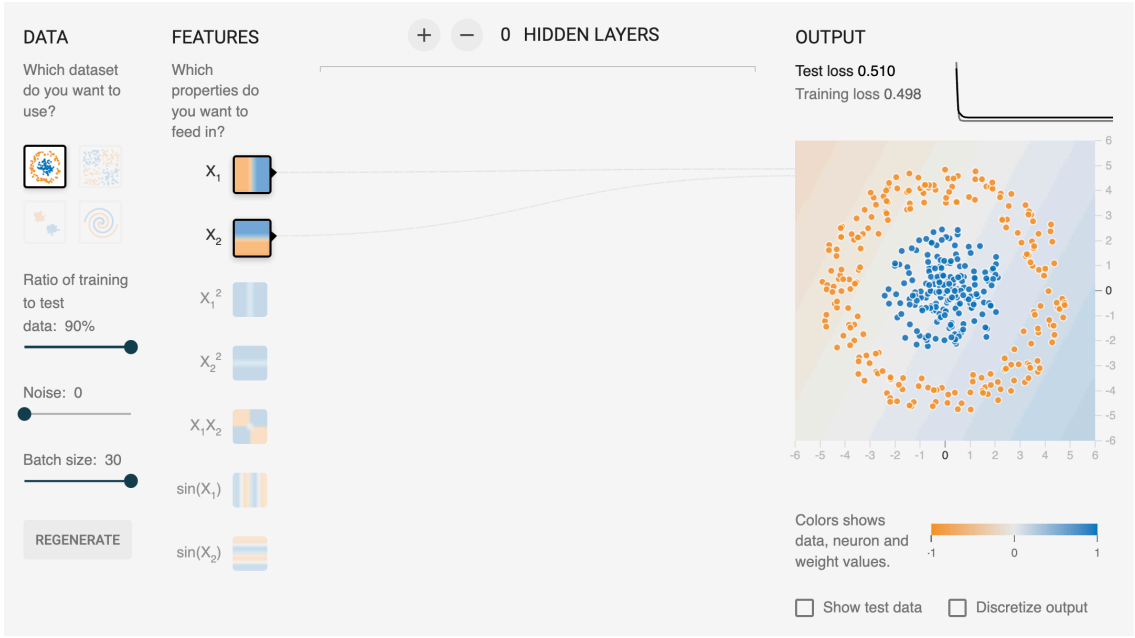
VC dimensions of NNs

What's the VC dimension for a neural network? What is the "model capacity" for a neural network?

Can a neural network shatter 4 points?



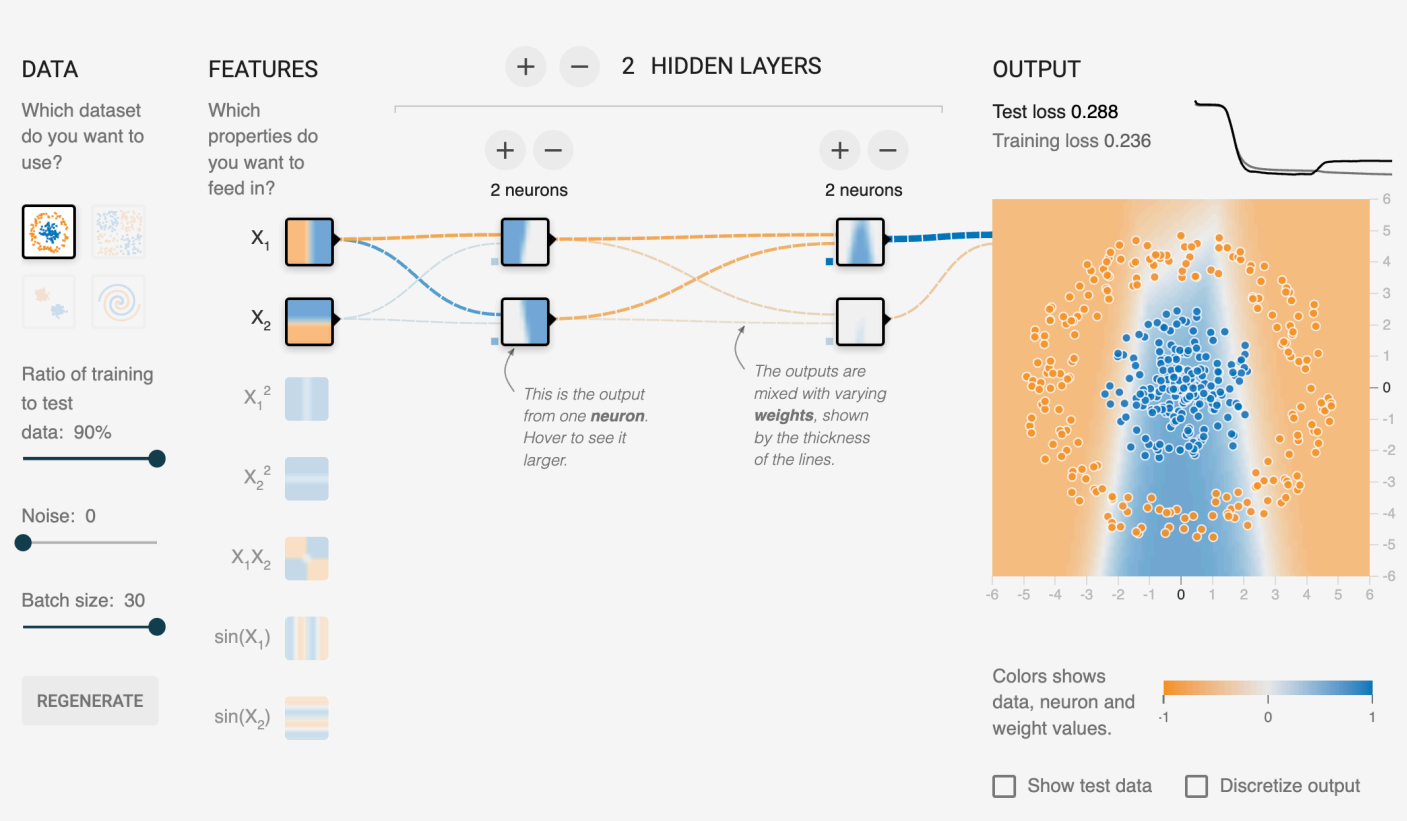
Can a neural network learn a pattern like this?



VC dimensions of NNs

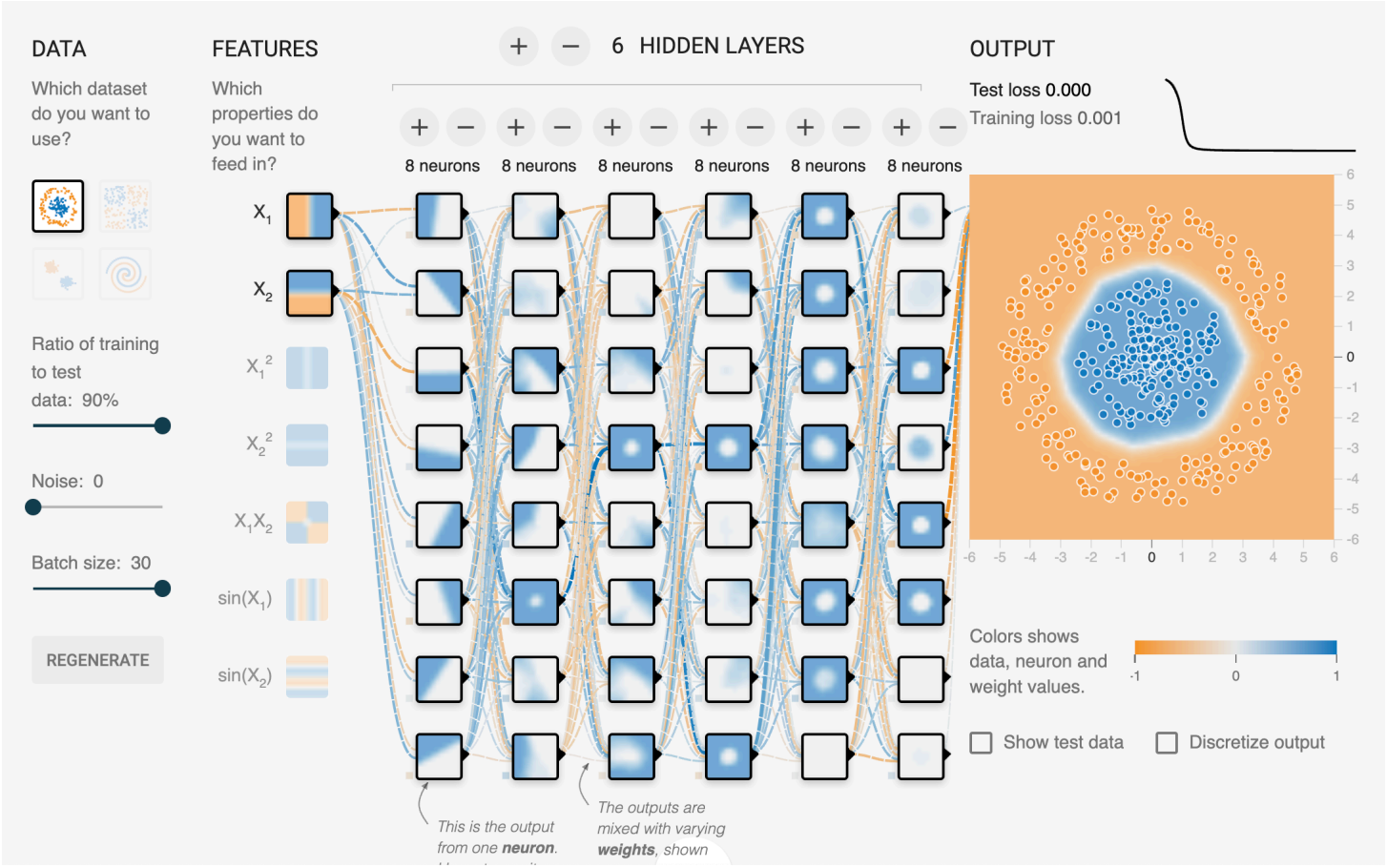
The VC dimension of a small NN is bigger than that of a line…

but still not *that* big.



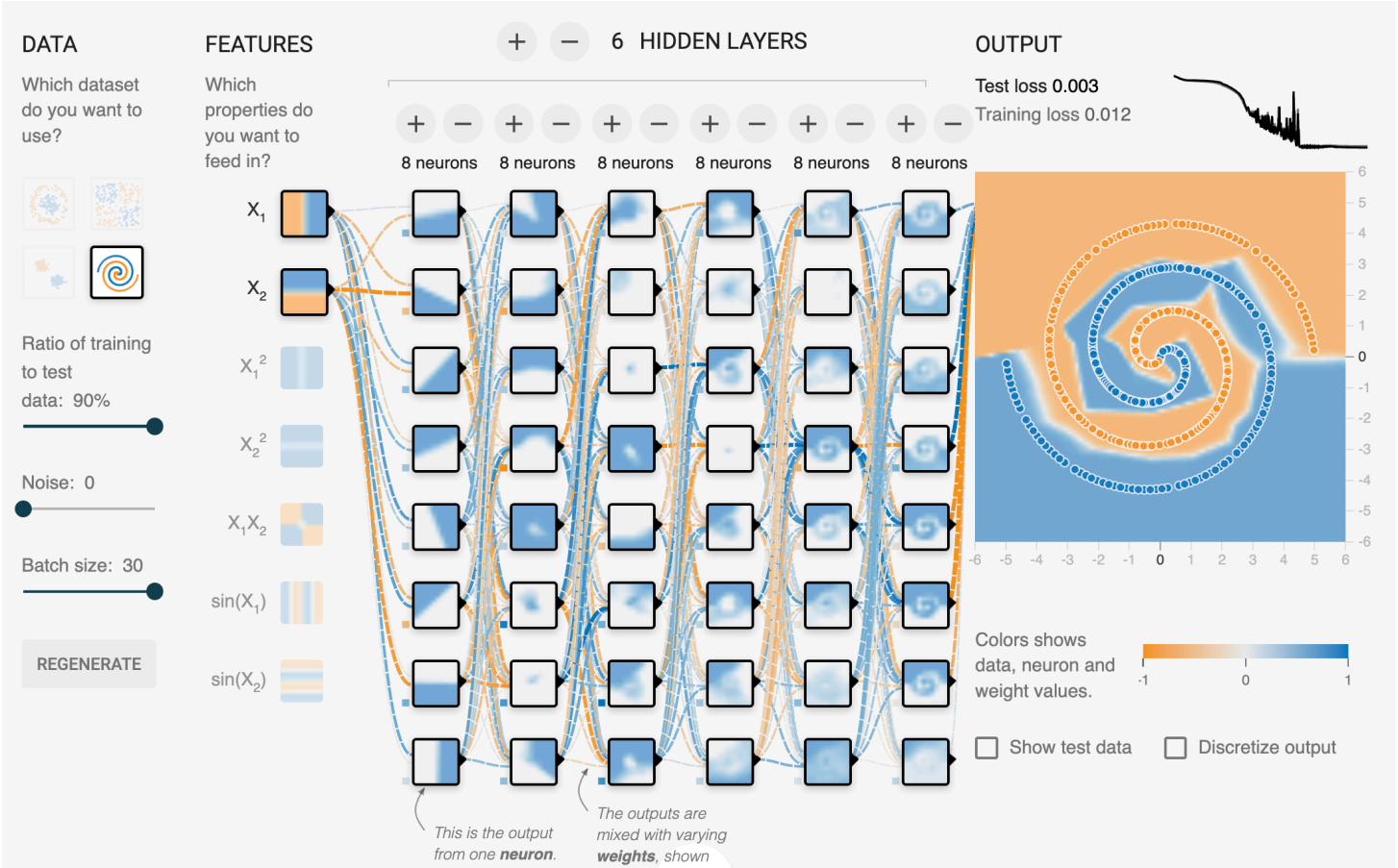
VC dimensions of NNs

The VC dimension of a deep NN is even larger.



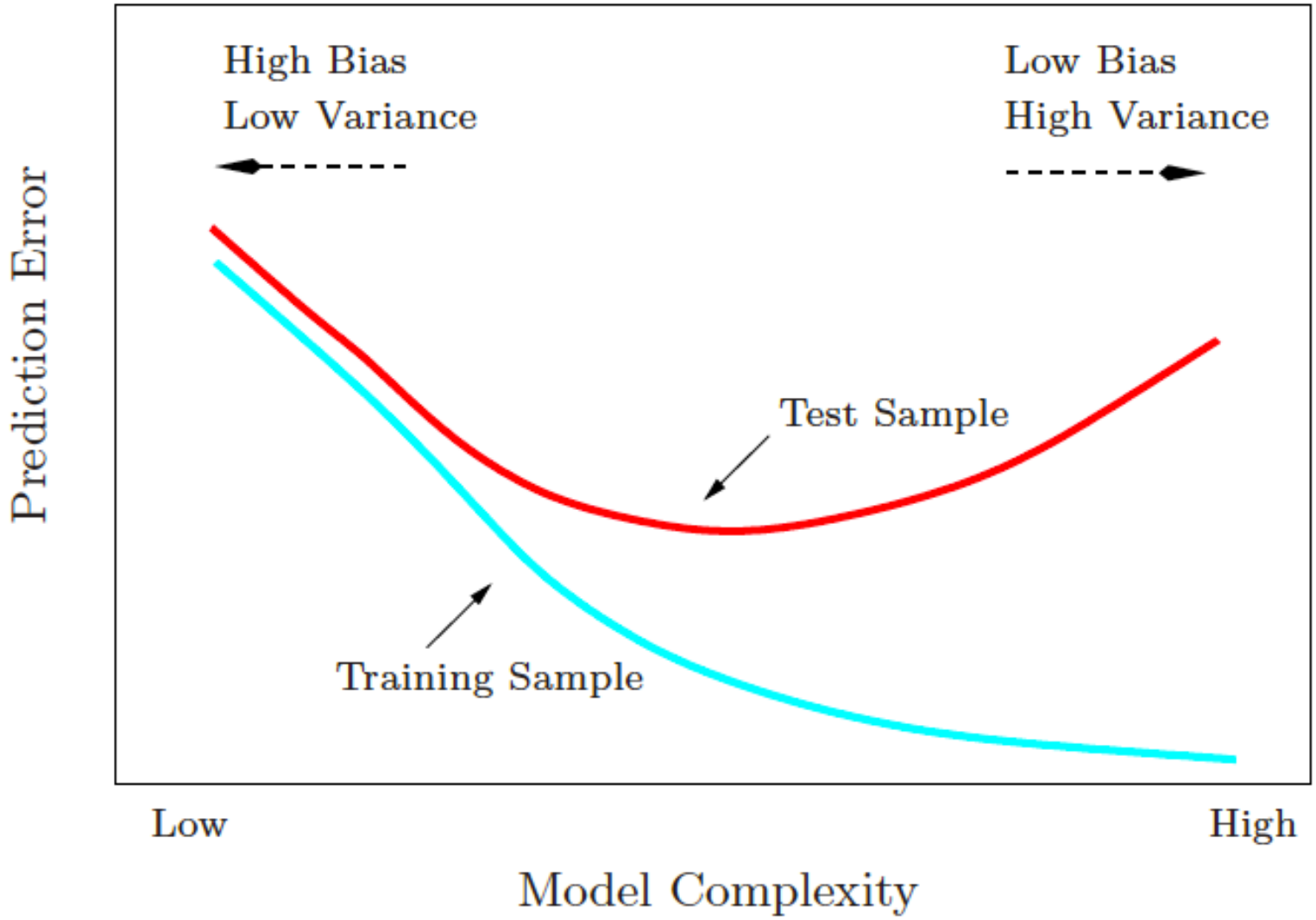
VC dimensions of NNs

The VC dimension of a deep NN is even larger.



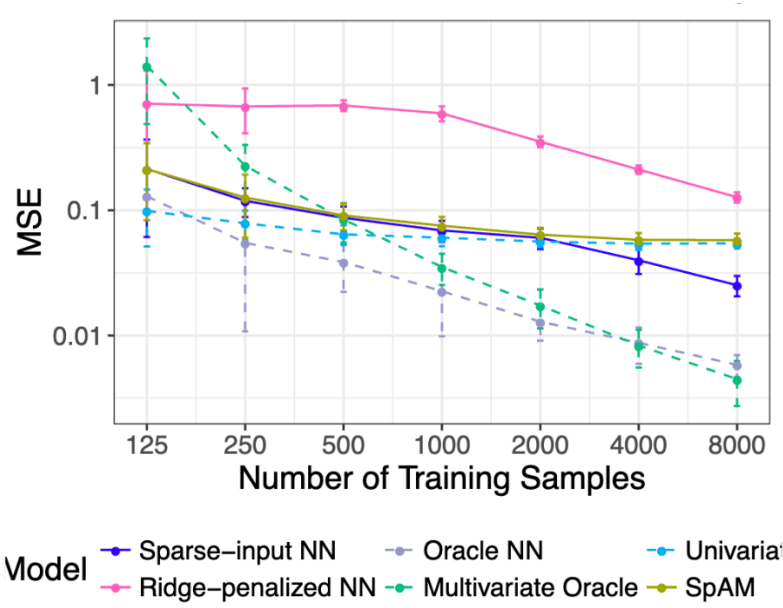
Bias variance tradeoff

Where do neural networks live?



The flip side: Overfitting

- The VC dimension of NNs is large, so it can:
  - Learn extremely complicated functions
  - Memorize your training data.
- This is a major reason why NNs perform poorly on tabular data compared to other ML models.
- To train RNs that generalize well, you need to carefully take into account the **bias variance tradeoff**!

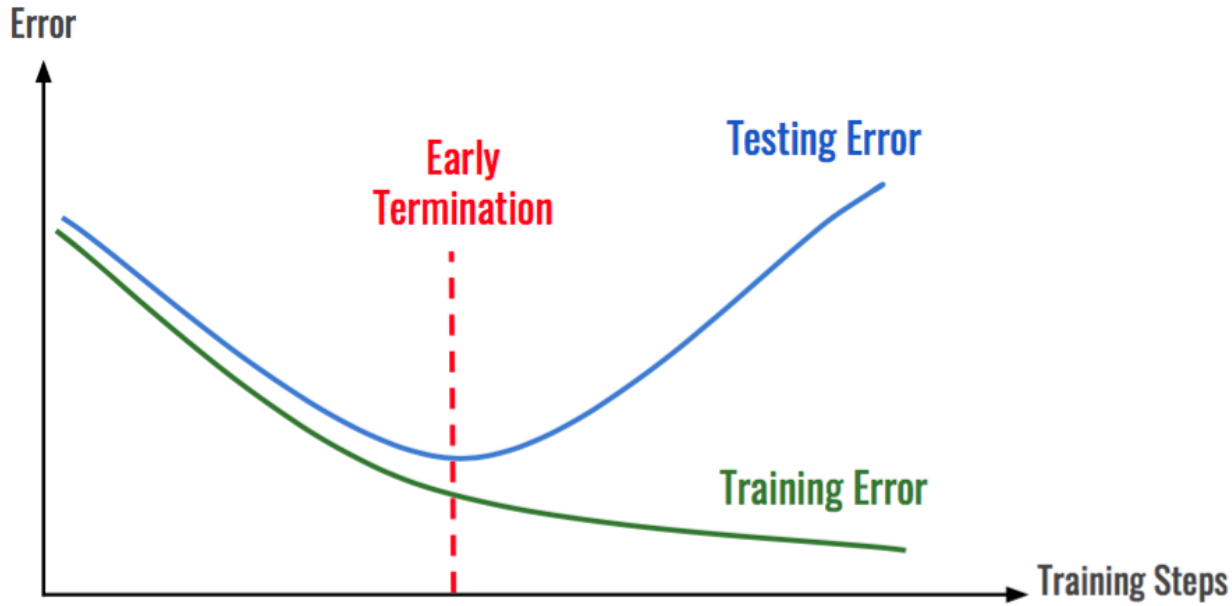


Feng and Simon 2017

Regularization

- Various methods have been developed to mitigate overfitting of NNs:
  - Early stopping
  - Dropout
  - Ensembling
  - Bayesian NNs
  - Penalization (also known as weight decay)
  - Data augmentation (e.g. image rotation)

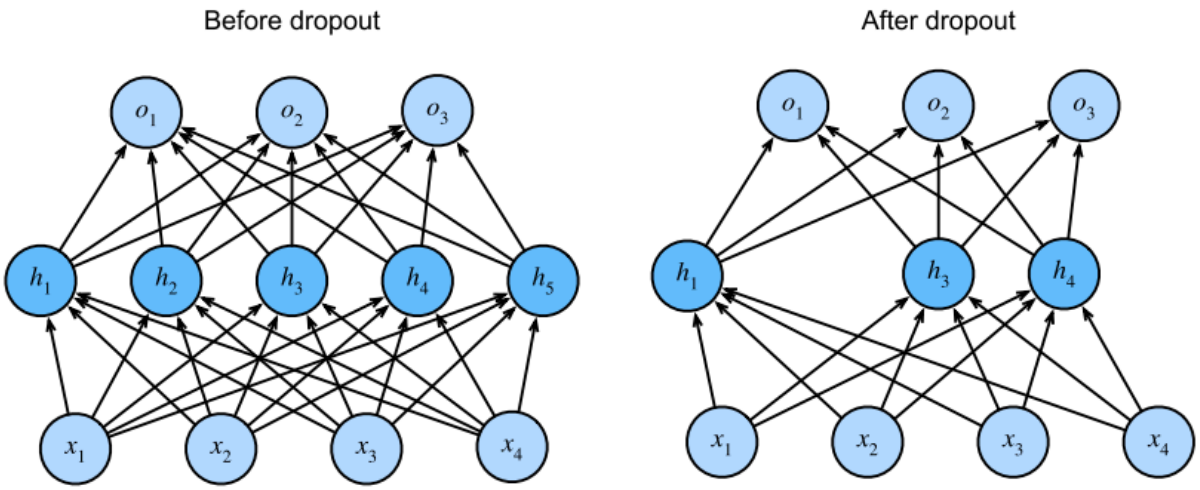
Early stopping



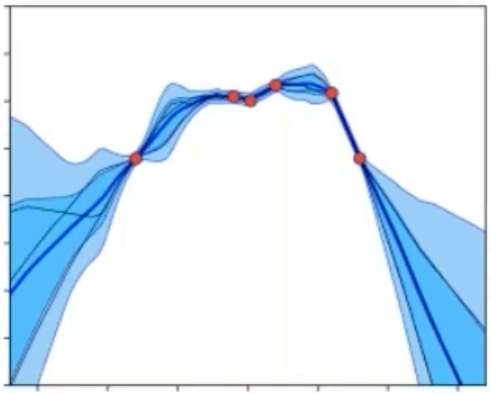
Dropout

- For each iteration during training, zero out some random subset of nodes in each layer. Debias each layer by normalizing by the fraction of nodes that were not dropped out.
- With dropout probability p, each intermediate activation h is replaced by a random variable h as follows:

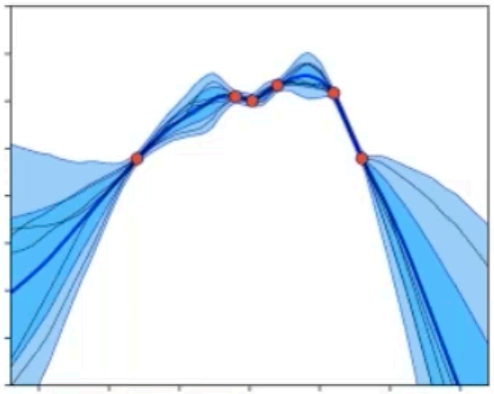
$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$



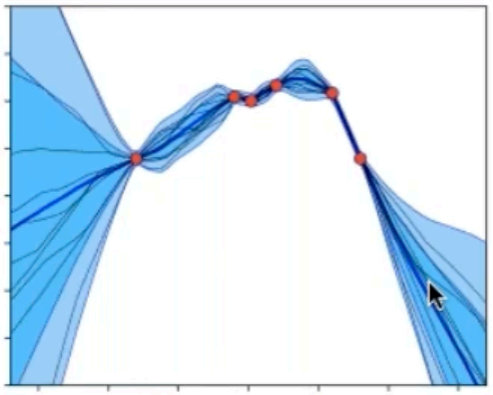
Ensembling



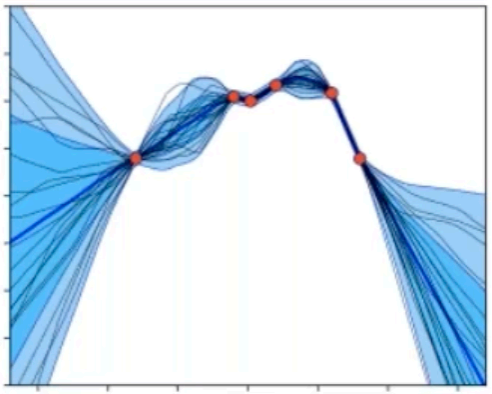
3x NNs



5x NNs

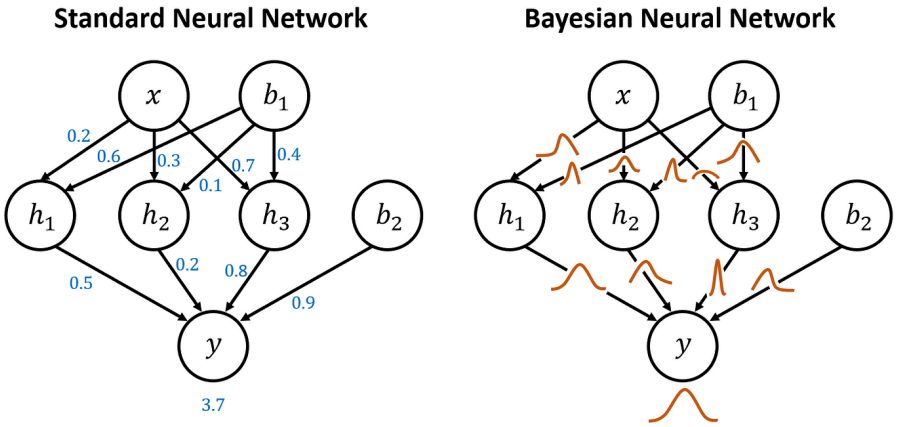


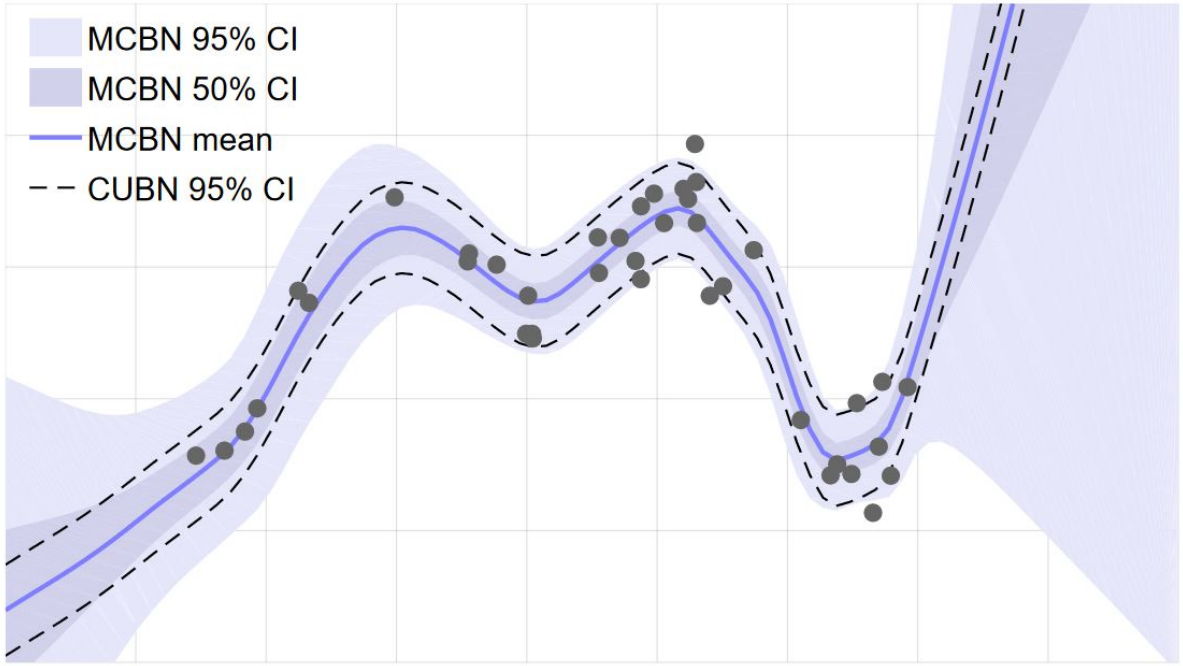
10x NNs



20x NNs

Bayesian NNs



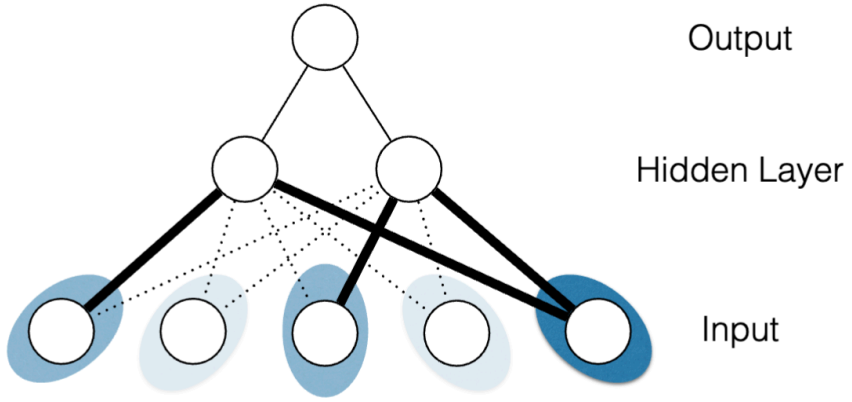


Penalization

$$\underset{\eta \in \mathbb{R}^{mp+2m+1}}{\arg\min} \frac{1}{n} \sum_{i=1}^{n} \ell\left(y_i, f_{\eta}(\boldsymbol{x}_i)\right) + \lambda_0 \|\boldsymbol{\beta}\|_2^2 + \lambda \sum_{j=1}^{p} \Omega_{\alpha}(\boldsymbol{\theta}_{(j)})$$

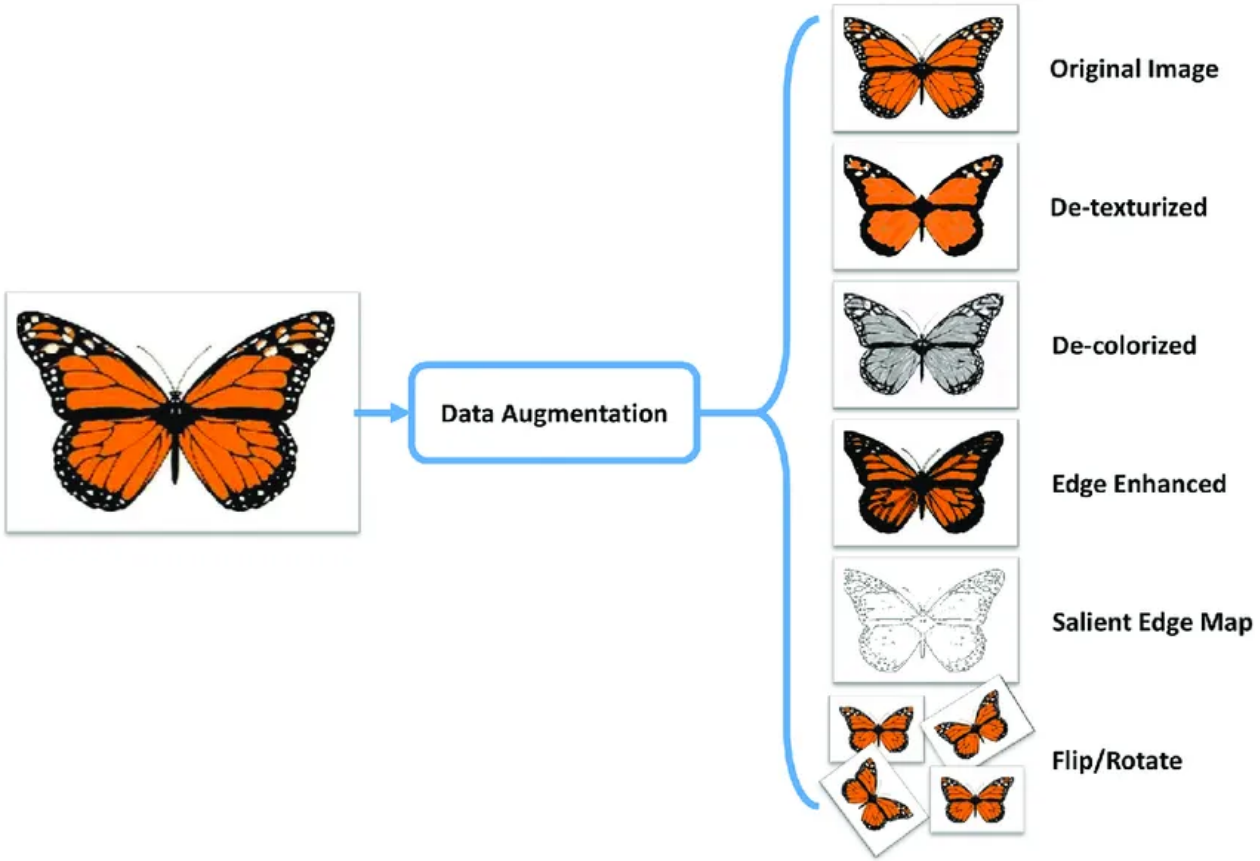
where $\alpha \in [0, 1]$ and $\Omega_{\alpha}(\boldsymbol{\theta}) = (1 - \alpha)\|\boldsymbol{\theta}\|_1 + \alpha\|\boldsymbol{\theta}\|_2$ is the sparse group lasso.

- L2 penalization
- L1 penalization
- Elastic net penalization
- Sparse group lasso



Data Augmentation

- If your data has known invariances, you can perturb your data with respect to these invariances to increase sample size.



Lecture Outline

- Deep learning
- Dense neural networks
  - Backward Propagation
  - Optimization: From Stochastic Gradient Descent to Adam
  - Numerical Stability and Initialization
  - Bias-variance tradeoff
- **Pytorch**

- PyTorch is an open-source machine learning library developed by Facebook's AI Research lab provides a number of key features:
  - Let's you build the computational graph
  - Automatic differentiation system
  - GPU acceleration
  - Distributed training

You'll no longer have to derive derivatives again!

- It has become the tool of choice for most ML researchers.
- There are lots of great guides online!
  - https://pytorch.org/tutorials/beginner/basics/intro.html

Introduction to PyTorch

Tensors in PyTorch have very similar operations as arrays in numpy

```
import torch
import numpy as np
```

```
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
```

Standard numpy-like indexing and slicing:

```
tensor = torch.ones(4, 4)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[..., -1]}")
tensor[:,1] = 0
print(tensor)
```

Arithmetic operations

```
# This computes the matrix multiplication between two tensors. y1, y2, y3
will have the same value
# ``tensor.T`` returns the transpose of a tensor
y1 = tensor @ tensor.T
y2 = tensor.matmul(tensor.T)

y3 = torch.rand_like(y1)
torch.matmul(tensor, tensor.T, out=y3)
```

Introduction to PyTorch

- PyTorch creates a computational graph that allows us to perform automatic differentiation.
  - https://pytorch.org/blog/overview-of-pytorch-autograd-engine/
  - https://pytorch.org/blog/computational-graphs-constructed-in-pytorch/

Other useful references

- **Stanford CS229 notes:**
  - **Deep learning chapter**: https://cs229.stanford.edu/lectures-spring2022/main_notes.pdf